Developer Guide

Barcode-Driven Engine
2026 R1

Acumatica
The Cloud ERP

Contents

Copyright 4

Barcode-Driven Engine Guide 5

Preparing a Test Instance 6

Test Instance: General Information 6

Test Instance: To Deploy an Instance 6

Creating a Barcode Scan Class 8

Barcode Scan Class: General Information 8

Barcode Scan Class: To Create a Barcode Scan Class 10

Creating a Barcode Scan Mode 14

Barcode Scan Mode: General Information 14

Barcode Scan Mode: To Define the Required Properties 17

Implementing Barcode Scan States 20

Barcode Scan States: General Information 20

Barcode Scan States: To Create the Set of Scan States 21

Barcode Scan States: Input State 22

Barcode Scan States: To Create the Input State 24

Barcode Scan States: Confirmation State 28

Barcode Scan States: To Create the Confirmation State 29

Implementing Barcode Scan Transitions 34

Barcode Scan Transitions: General Information 34

Barcode Scan Transitions: To Implement Transitions 35

Barcode Scan Transitions: To Implement a Simple Transition Flow 36

Defining the List of Barcode Scan Commands 39

Barcode Scan Commands: General Information 39

Barcode Scan Commands: To Define the List of Commands 40

Barcode Scan Commands: To Implement a Custom Command 41

Barcode Scan Commands: To Add Quantity Support 44

Defining the Set of Scan Mode Redirects 46

Scan Mode Redirects: General Information 46

Scan Mode Redirects: To Define the List of Redirects 48

Resetting a Barcode Scan Mode 51

Reset of Barcode Scan Mode: General Information 51

Reset of Barcode Scan Mode: To Define the Resetting Logic 51

Creating a Custom Barcode-Driven Form 54

Custom Barcode-Driven Form: General Information....54
Custom Barcode-Driven Form: To Create an ASPX Page.... 55
Extending Scan Components.... 61
Extension of Scan Components: General Information.... 61
Extension of Scan Components: Method Interceptors.... 62
Extension of Scan Components: Scan Extensions.... 64
Extension of Scan Components: To Extend Predefined Scan States for a Custom Form.... 67
Extension of Scan Components: Customization of Components....70
Extension of Scan Components: To Change the Order of Value Input....75
Extension of Scan Components: To Add a New Scan Command for an Existing Form.... 78
Extension of Scan Components: To Implement Additional Entity Validation.... 81
Extension of Scan Components: To Add an Alternate Way to Search for Entities.... 84
Customizing a Barcode-Driven Form.... 88
Customization of a Barcode-Driven Form: General Information.... 88
Customization of a Barcode-Driven Form: Barcode Scan Class....89
Customization of a Barcode-Driven Form: To Handle Input Without Component Usage....89

Copyright

© 2026 Acumatica, Inc.

ALL RIGHTS RESERVED.

No part of this document may be reproduced, copied, or transmitted without the express prior consent of Acumatica, Inc.

3075 112th Avenue NE, Suite 200, Bellevue, WA 98004, USA

Restricted Rights

The product is provided with restricted rights. Use, duplication, or disclosure by the United States Government is subject to restrictions as set forth in the applicable License and Services Agreement and in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 or subparagraphs (c)(1) and (c)(2) of the Commercial Computer Software-Restricted Rights at 48 CFR 52.227-19, as applicable.

Disclaimer

Acumatica, Inc. makes no representations or warranties with respect to the contents or use of this document, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Acumatica, Inc. reserves the right to revise this document and make changes in its content at any time, without obligation to notify any person or entity of such revisions or changes.

Trademarks

Acumatica is a registered trademark of Acumatica, Inc. HubSpot is a registered trademark of HubSpot, Inc. Microsoft Exchange and Microsoft Exchange Server are registered trademarks of Microsoft Corporation. All other product names and services herein are trademarks or service marks of their respective companies.

Software Version: 2026 R1

Last Updated: 03/15/2026

Barcode-Driven Engine Guide

With Acumatica's barcode-driven engine, you can convert an Acumatica ERP form, which is operated via a mouse and keyboard, into a console-type experience for a barcode-driven form. A user interacts with this form mostly via text commands that the user enters into a special field of the form manually or by scanning a barcode. The barcode-driven form guides the user through a certain input route by showing input prompts. This console-type experience is useful when you need to simplify the work with a particular form to the level of sequential text inputs.

In this guide, you can find information about how to customize existing Acumatica ERP forms that use the barcode-driven engine and how to implement custom barcode-driven forms.

You can find the code and customization project of the activities described in this guide in the *Help-and-Training-Examples* repository on GitHub.

Preparing a Test Instance

The topics of this chapter describe how to deploy an Acumatica ERP instance that you can use to test the examples of these guide.

Test Instance: General Information

In this chapter, you will learn how to deploy an Acumatica ERP test instance that contains custom and customized forms. You can use this instance to complete a training course or test a scenario described in this guide. *Custom forms* are forms that are created entirely from scratch by a customizer. *Customized forms* are forms that are based on predefined forms in Acumatica ERP, but are modified by a customizer to fit a specific business process.

Learning Objectives

In this chapter, you will learn how to do the following:

- Prepare the environment
- Deploy an Acumatica ERP test instance

Applicable Scenarios

You deploy an Acumatica ERP test instance by using the instructions in this chapter in the following cases:

- You want to test the activities described in this guide.
- You need to complete a training course.

Deploying a Test Instance

You can use the Acumatica ERP Configuration wizard to deploy instances based on predefined datasets. It also makes it possible to deploy test instances that contain custom and customized forms.

In the Acumatica ERP Configuration wizard, you click **Deploy a New Acumatica ERP Instance for T-Series Developer Courses** and select the applicable training course in the list. The instance that you are preparing contains the data that is required to complete the activities of this guide. The instance can also include the published customization project.

Test Instance: To Deploy an Instance

This activity will walk you through the process of preparing the environment and deploying an Acumatica ERP instance that you can use to perform the activities of this guide.

Story

Suppose that you are preparing to create a custom barcode-driven Acumatica ERP form. Before you can start the customization, you need to deploy an Acumatica ERP instance by using the Acumatica ERP Configuration wizard.

Process Overview

You will prepare the environment that you will use to customize Acumatica ERP. Then you will deploy an Acumatica ERP instance with the data for the *T210 Customized Forms and Master-Details Relationships* course.

> This instance will contain the data that you can use for testing of activities in this guide, the *PhoneRepairShop* customization project, and the source code of the extension library of the customization project.

Step 1: Preparing the Environment

Before you begin deploying the needed Acumatica ERP instance, do the following:

1. Make sure that the environment you're going to use conforms to the *System Requirements for the Acumatica ERP Installation*.
2. Make sure that the Web Server (IIS) features listed in *Configuration of IIS Web Server Features* are turned on.
3. Install the Acuminator extension for Visual Studio.
4. Install Acumatica ERP. On the Main Software Configuration page of the Acumatica ERP Setup wizard, select the **Install Acumatica ERP** and **Install Debugger Tools** check boxes.

   > If you've already installed Acumatica ERP without the debugger tools, you should uninstall it and install it again with the **Install Debugger Tools** check box selected. The reinstallation of Acumatica ERP doesn't affect existing Acumatica ERP instances. You can also install the Acumatica ERP Tools separately. For details, see *Acumatica ERP Installation On-Premises: To Install the Acumatica ERP Tools (Optional)*.

Step 2: Deploying an Acumatica ERP Instance

You deploy an Acumatica ERP instance and configure it as follows:

1. Open the Acumatica ERP Configuration wizard, and do the following:
   a. Click **Deploy a New Acumatica ERP Instance for T-Series Developer Courses**.
   b. On the Database Configuration page, make sure that the name of the database is *PhoneRepairShop*.
   c. On the Instance Configuration page, do the following:
      a. In the **Local Path to the Instance** box, select a folder that is outside of the `C:\Program Files (x86)`, `C:\Program Files`, and `C:\Users` folders. (We recommend that you store the website folder outside of these folders to avoid an issue with permission to work in these folders when you perform customization of the website.)
      b. In the **Training Course** box, select *T210 Customized Forms and Master-Details Relationships*.

   The system creates a new Acumatica ERP instance, adds a new tenant, loads the data to it, and publishes the customization project.
2. Make sure a Visual Studio solution is available in the `App_Data\Projects\PhoneRepairShop` folder of the Acumatica ERP instance folder. This is the solution of the extension library in which you will implement the customization in the activities of this guide.

Creating a Barcode Scan Class

In this chapter, you will learn about a barcode scan class, which is the main class of a barcode-driven form. You will also create a barcode scan class.

Barcode Scan Class: General Information

A barcode scan class is the main class of a barcode-driven form.

Learning Objectives

In this chapter, you will learn how to do the following:

- Create a barcode scan class
- Make the barcode scan class usable

Applicable Scenarios

You create a barcode scan class if you are creating a custom barcode-driven Acumatica ERP form.

Core Class

`BarcodeDrivenStateMachine<TSelf, TGraph>` is the core class of the Acumatica barcode-driven engine. This is a generic graph extension that connects all components of the barcode-driven engine. The class implements the `IBarcodeDrivenStateMachine` interface and inherits from the `PXGraphExtension<TGraph>` class. All components can access the core class via their `Basis` property.

> All components of the barcode-driven engine are available in the `PX.BarcodeProcessing.dll` assembly. To make the barcode-driven engine usable, you need to add a reference to this assembly in the project of your Acumatica ERP extension library and add a `using` directive for the `PX.BarcodeProcessing` namespace to the source files.

The core class contains the components shown in the following diagram. For details about the properties and methods of the core class, see *BarcodeDrivenStateMachine<TSelf,TGraph> Class*.

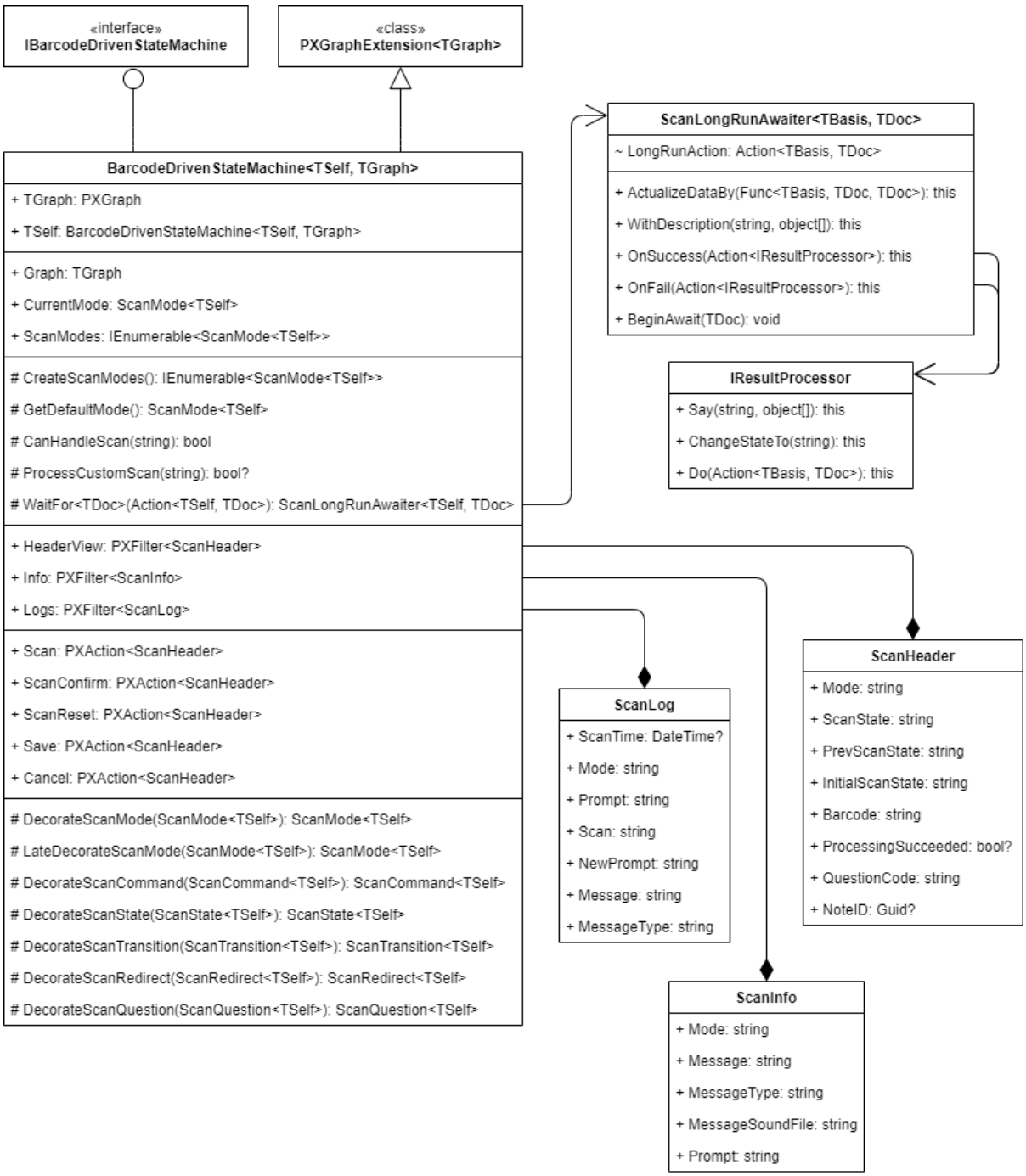


Figure: The core class

Barcode Scan Class

While creating a barcode-driven form, you introduce a barcode scan class as a descendant of the core class (that is, `BarcodeDrivenStateMachine<TSelf, TGraph>`). The barcode scan class is a graph extension that meets the following requirements:

- The class is enclosed with itself in the `TSelf` type parameter. All the components of the barcode scan class will have the `TScanBasis` type parameter set to the class passed into the `TSelf` type parameter.
- The class has a host graph in the `TGraph` type parameter. For details about the host graph, see the next section of this topic.

You then implement abstract members of the barcode scan class.

As is true of any other graph extension, the barcode scan class can contain views, actions, event handlers, and any other types of members. However, to make the barcode-driven part of the extension work, you must implement the `CreateScanModes()` abstract method. This method returns the set of scan modes available on the barcode-driven form. The scan mode creates and maintains scan states, transitions between them, available commands, and user dialogs. Scan modes are parameterized with the current barcode scan class type.

Host Graph

The host graph is a supplementary graph that connects the barcode scan class with the target graph to which the barcode-driven engine functionality is applied.

> ⚠ Do not apply a barcode scan class directly to a target graph because this can significantly change the way the form works and break the graph's standard workflow.

You can use one of the following ways to introduce a host graph:

- By making the host graph an empty descendant of an existing graph.

  With this approach, you reuse the business logic of the existing graph, while the barcode scan class is applied only to that descendant and not to the original graph. Therefore, both versions of the graph (the original one and the one with the barcode scan functionality) are available in the system.

  The barcode scan class and all its components can use the inherited functionality of the host graph by accessing the `Graph` property of the barcode scan class.
- By making the host graph a new empty graph (that is, a direct descendant of the `PXGraph<T>` class).

  This approach is useful when the original graph you want to utilize is pretty expensive to use, so you want to instantiate it manually and only when it is actually needed. That is, you want to instantiate it manually only when the barcode scan class applies the changes accumulated during scan cycles.

Barcode Scan Class: To Create a Barcode Scan Class

This activity will walk you through the process of creating a barcode scan class.

Story

Suppose that you need to create a barcode-driven version of the *Physical Inventory Review* (IN305000) form. The form should have only one scan mode. For this form, you need to define a barcode scan class and configure this mode.

Process Overview

You will define the `INScanCount` barcode scan class as a descendant of the `WarehouseManagementSystem<TSelf,TGraph>` base scan class.

In the `INScanCount` class, you will create a host class as an empty descendant for `INPICountEntry`, the graph object of the *Physical Inventory Review* (IN305000) form.

You will then implement the `CreateScanModes() abstract` method of the base class to make the barcode scan class usable. In the method, you will use the `yield return` construction to avoid array creation.

System Preparation

Before you begin performing the steps of this activity, you need to prepare an Acumatica ERP instance and the *PhoneRepairShop* customization project by performing the *Test Instance: To Deploy an Instance* prerequisite activity.

Step 1: Creating a Barcode Scan Class

To create a barcode scan class, do the following:

1. In the `PhoneRepairShop_Code` Visual Studio project, add a reference to the `PX.BarcodeProcessing.dll` file.
2. Create the `INScanCount` class and make it `public`.
3. Add the `using` directives shown in the following code to the file.

```
using PX.BarcodeProcessing;
using PX.Objects.IN;
using PX.Objects.IN.WMS;
```

4. In the `INScanCount` class, add a host class to apply the barcode scan class to, as shown in the following code.

```
public class INScanCount
{
    public class Host : INPICountEntry { }
}
```

5. To simplify referencing the nested components of the barcode scan class, add the following line before the `INScanCount` class declaration. The `WarehouseManagementSystem<TSelf,TGraph>` class is a descendant of the `BarcodeDrivenStateMachine<TSelf, TGraph>` class, which contains warehouse-related logic and components.

```
using WMSBase = WarehouseManagementSystem<INScanCount, INScanCount.Host>;
```

6. Derive the `INScanCount` class from `WMSBase`. The following code shows the result of this step.

```
using PX.BarcodeProcessing;
using PX.Objects.IN;
using PX.Objects.IN.WMS;

namespace PhoneRepairShop
{
    using WMSBase = WarehouseManagementSystem<INScanCount, INScanCount.Host>;
    public class INScanCount: WMSBase
    {
        public class Host : INPICountEntry { }
    }
}
```

7. Fix or suppress the *PX1016* error that is displayed by Acuminator.

Step 2: Making the Barcode Scan Class Usable

To make the barcode scan class usable, do the following:

1. Add the `using` directives shown in the following code to the file.

```
using System.Collections.Generic;
using PX.Data;
```

2. To make the barcode-driven part of the graph extension work, implement the `CreateScanModes()` abstract method as follows.

```
public class INScanCount : WMSBase
{
    ...

    protected override IEnumerable<ScanMode<INScanCount>> CreateScanModes()
    {
        yield return new CountMode();
    }
}
```

> You will implement the `CountMode` scan mode in *Barcode Scan Mode: To Define the Required Properties*.

3. To facilitate access to the important members of the host graph, add the following members.

```
public class INScanCount : WMSBase
{
    ...

    public virtual INPIHeader Document => DocumentView.Current;
    public virtual PXSelectBase<INPIHeader> DocumentView => Graph.PIHeader;
    public virtual PXSelectBase<INPIDetail> Details => Graph.PIDetail;

    ...
}
```

4. Add the following supplementary method, which will be reused in further implementation of the custom barcode-driven form.

```
public class INScanCount : WMSBase
{
    ...

    protected virtual bool IsDocumentStatusEditable(string status) =>
        status == INPIHdrStatus.Counting;
    ...
}
```

5. To define when the document with which the barcode scan class works can be edited by the barcode processing, override the `DocumentIsEditable` member of the `WMSBase` class.

```
public class INScanCount : WMSBase
{
    ...

    public override bool DocumentIsEditable =>
        base.DocumentIsEditable && IsDocumentStatusEditable(Document.Status);

    ...
```

```
}
```

Creating a Barcode Scan Mode

In this chapter, you will learn about a barcode scan mode, which is the main component of a barcode-driven Acumatica ERP form. You will also learn how to define the required properties of the barcode scan mode.

Barcode Scan Mode: General Information

A barcode scan mode is the main scan component of the Acumatica barcode-driven engine.

Learning Objectives

In this chapter, you will learn how to define the required properties of the barcode scan mode.

Applicable Scenarios

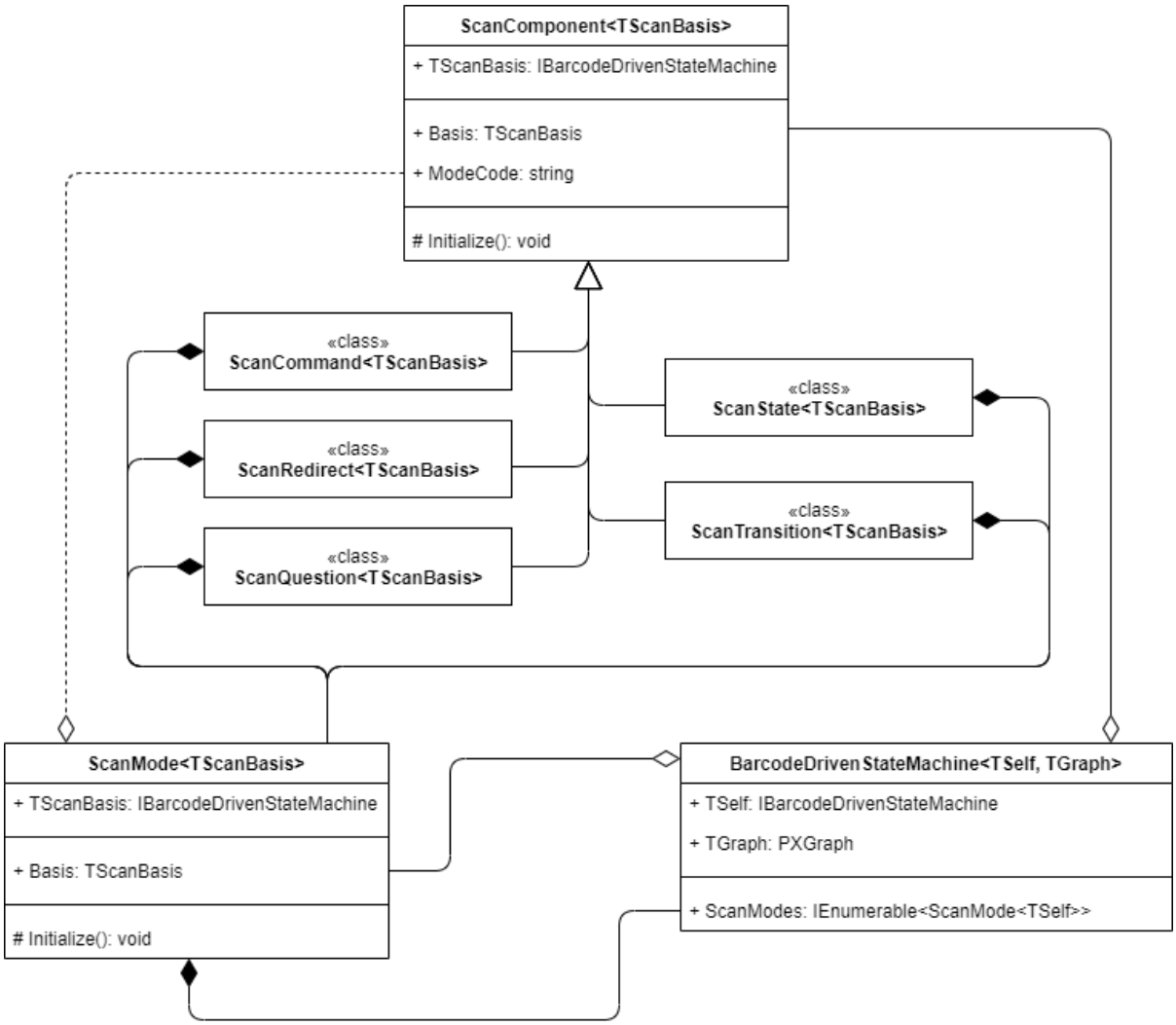
You create a barcode scan mode in the following cases:

- You are creating a custom barcode-driven form. You have defined a barcode scan class for this form and created an instance of the descendant of the `ScanMode` class in the `CreateScanModes()` method implementation. Now you need to define this descendant itself.
- You need to add a scan mode to an existing barcode-driven form. You have created a scan extension of a barcode scan class and overridden the `CreateScanModes()` method to return the new scan mode, as described in *Extension of Scan Components: Customization of Components*. Now you need to implement this new mode.

Barcode Scan Components

All barcode scan components except `ScanMode<TScanBasis>` are descendants of the `ScanComponent<TScanBasis>` class. `ScanComponent<TScanBasis>` provides access to the barcode scan class via its `Basis` property. The class also contains information about the mode to which it belongs.

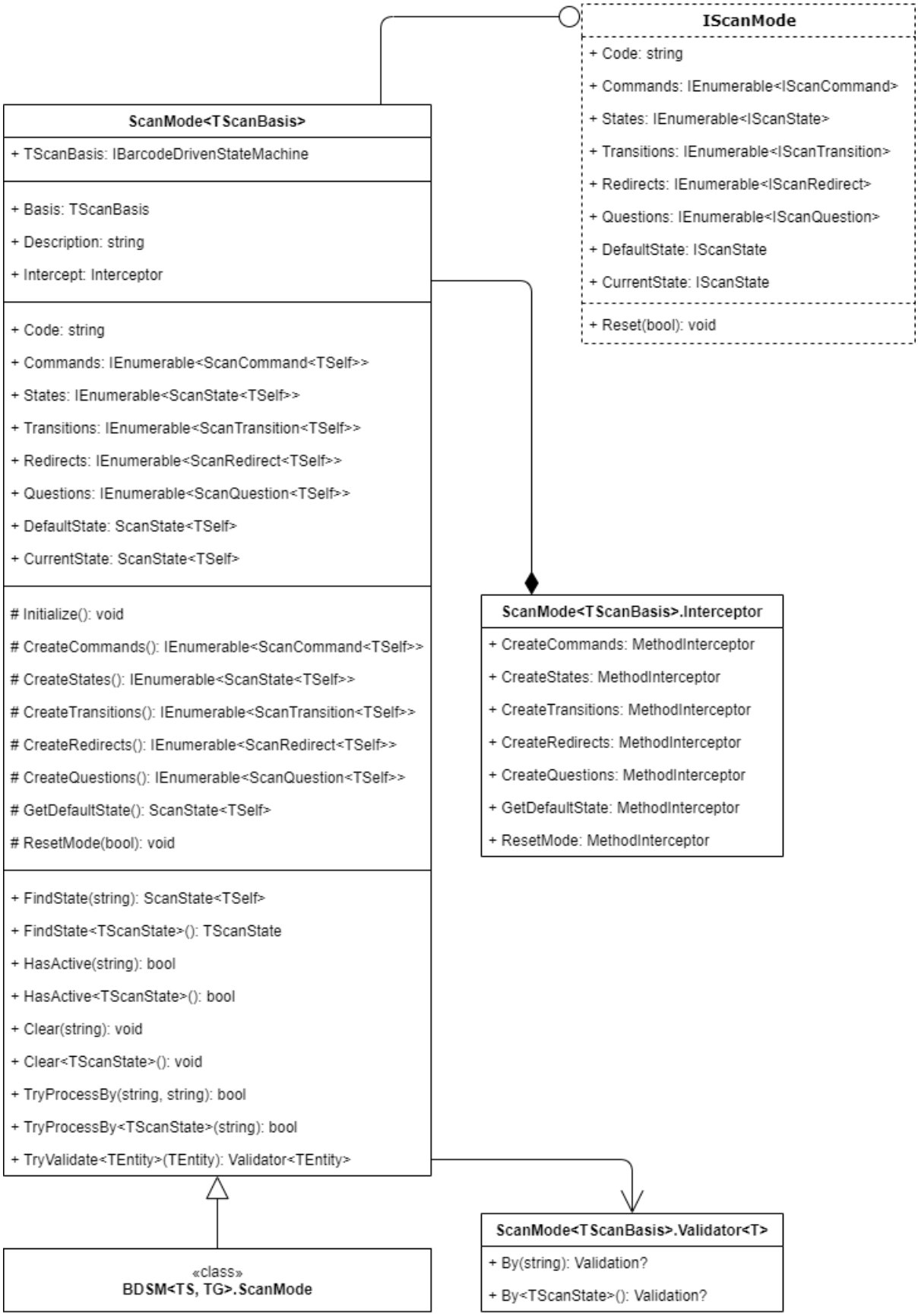
The following diagram shows all barcode scan components and the relations between them. For details on the properties of the components, see the API Reference.



Scan Mode

`ScanMode<TScanBasis>` is the main component of the `BarcodeDrivenStateMachine` and the base class for all modes. It provides access to all other components and defines their structure and the relationships between them.

The following diagram shows the structure of the scan mode. For details on the properties and methods, see *ScanMode<TScanBasis> Class*.


IScanMode
+ Code: string
+ Commands: IEnumerable<IScanCommand>
+ States: IEnumerable<IScanState>
+ Transitions: IEnumerable<IScanTransition>
+ Redirects: IEnumerable<IScanRedirect>
+ Questions: IEnumerable<IScanQuestion>
+ DefaultState: IScanState
+ CurrentState: IScanState
+ Reset(bool): void
ScanMode<TScanBasis>
+ TScanBasis: IBarcodeDrivenStateMachine
+ Basis: TScanBasis
+ Description: string
+ Intercept: Interceptor
+ Code: string
+ Commands: IEnumerable<ScanCommand<TSelf>>
+ States: IEnumerable<ScanState<TSelf>>
+ Transitions: IEnumerable<ScanTransition<TSelf>>
+ Redirects: IEnumerable<ScanRedirect<TSelf>>
+ Questions: IEnumerable<ScanQuestion<TSelf>>
+ DefaultState: ScanState<TSelf>
+ CurrentState: ScanState<TSelf>
# Initialize(): void
# CreateCommands(): IEnumerable<ScanCommand<TSelf>>
# CreateStates(): IEnumerable<ScanState<TSelf>>
# CreateTransitions(): IEnumerable<ScanTransition<TSelf>>
# CreateRedirects(): IEnumerable<ScanRedirect<TSelf>>
# CreateQuestions(): IEnumerable<ScanQuestion<TSelf>>
# GetDefaultState(): ScanState<TSelf>
# ResetMode(bool): void
+ FindState(string): ScanState<TSelf>
+ FindState<TScanState>(): TScanState
+ HasActive(string): bool
+ HasActive<TScanState>(): bool
+ Clear(string): void
+ Clear<TScanState>(): void
+ TryProcessBy(string, string): bool
+ TryProcessBy<TScanState>(string): bool
+ TryValidate<TEntity>(TEntity): Validator<TEntity>
ScanMode<TScanBasis>.Interceptor
+ CreateCommands: MethodInterceptor
+ CreateStates: MethodInterceptor
+ CreateTransitions: MethodInterceptor
+ CreateRedirects: MethodInterceptor
+ CreateQuestions: MethodInterceptor
+ GetDefaultState: MethodInterceptor
+ ResetMode: MethodInterceptor
«class»
BDSM<TS, TG>.ScanMode
ScanMode<TScanBasis>.Validator<T>
+ By(string): Validation?
+ By<TScanState>(): Validation?

Recommendations for Scan Components

All scan components that you create should be either `abstract` classes or `sealed` classes. You can introduce an `abstract` component to provide a base implementation that is shared by multiple `sealed` components. We recommend this approach to prevent other developers from using inheritance to alter the behavior of the components defined in your code. Inheritance should be used only when the developer creates a new scan mode or a new barcode-driven Acumatica ERP form from scratch. To alter the behavior of scan components, the developer should use the method interception approach because this approach makes it possible to apply multiple customizations to one component. For details on this approach, see *Extension of Scan Components: Method Interceptors*.

> In the source code of Acumatica ERP, there are multiple exceptions to this approach in the definition of component classes. Most of them are the components defined in the `WarehouseManagementSystem<TSelf, TGraph>` class (such as `InventoryItemState`, `LocationState`, and `RemoveCommand`). These components can be used either without changes or by providing its descendant with mode-related modifications. These classes are neither `abstract` nor `sealed`. However, you should not use inheritance to customize their behavior. In customization scenarios, you should use only the method interception approach.

Because scan components are strictly bound to a specific barcode-driven form, you can nest these components within the barcode scan class or even within each other. For example, you can nest the following: scan modes within the barcode scan class; scan states, scan transitions, and scan commands within the scan mode; and scan extensions within the components that use them.

Implementation of a Scan Mode

You implement a scan mode as follows:

1. You define the required properties of the mode, as shown in *Barcode Scan Mode: To Define the Required Properties*.

   For a scan mode, you must define the mode identifier and a user-friendly description, which are the required properties. We recommend that you make the code value of the mode statically available, and also available as a C# constant and a BQL constant.

2. You implement the component creation methods, as described in the following chapters:

   - *Implementing Barcode Scan States*
   - *Implementing Barcode Scan Transitions*
   - *Defining the List of Barcode Scan Commands*
   - *Defining the Set of Scan Mode Redirects*

   The component creation methods are optional for implementation. By default, they return empty sets; however, in most cases, the default behavior would be insufficient to make the barcode scan mode usable.

3. You define the mode resetting logic, as specified in the *Resetting a Barcode Scan Mode* chapter.

Barcode Scan Mode: To Define the Required Properties

This activity will walk you through the implementation of the required properties of a scan mode.

Story

Suppose that you are creating a custom barcode-driven form. You have defined a barcode scan class for this form and created an instance of the descendant of the `ScanMode` class in the implementation of the

`CreateScanModes()` method. Now you need to define this descendant itself. You also need to define the constants for the values of the required properties of the scan mode and add these properties.

Process Overview

You will define the class for the scan mode and define its required properties. For the `Code` property, you will provide both the `string` and the BQL constant. For the `Description` property, you will use a localizable message.

System Preparation

Before you begin creating a scan mode, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.

Step: Defining the Required Properties of the Scan Mode

To define the required properties of the scan mode, do the following:

1. In the `INScanCount` class, define the class for the scan mode, as shown in the following code.

```
public class INScanCount : WMSBase
{
    ...

    public sealed class CountMode : ScanMode
    {
    }
}
```

2. In the `INScanCount.cs` file, add the `using` directives shown in the following code.

```
using PX.Common;
using PX.Data.BQL;
```

3. Define the `string` constant and the BQL constant for the scan mode identifier, as shown below.

```
public class INScanCount : WMSBase
{
    ...

    public sealed class CountMode : ScanMode
    {
        public const string Value = "INCO";
        public class value : BqlString.Constant<value>
        {
            public value() : base(CountMode.Value) { }
        }

        public override string Code => Value;
    }
}
```

4. Define the localizable message for the name of the scan mode, as shown in the following code.

```
public class INScanCount : WMSBase
{
    ...

    public sealed class CountMode : ScanMode
    {
        ...
        public override string Description => Msg.Description;

        [PXLocalizable]
        new public abstract class Msg
        {
            public const string Description = "Scan and Count";
        }
    }
}
```

Implementing Barcode Scan States

In this chapter, you will learn about the implementation of barcode scan states, which contain the logic for handling a non-fixed input on a barcode-driven form. This input might include, for example, a barcode, quantity, or date.

Barcode Scan States: General Information

`ScanState<TScanState>` is a class for a barcode scan state. This class represents a component that contains the logic for handling a non-fixed input (such as a barcode, quantity, or date) of a barcode-driven Acumatica ERP form.

Learning Objectives

In this chapter, you will learn how to do the following:

- Create a set of scan states of a scan mode
- Implement the input scan state
- Implement the confirmation scan state

Applicable Scenarios

You create scan states in the following cases:

- You have created a scan mode on a custom form and need to implement the set of scan states of this mode.
- You have added a new scan mode on an existing barcode-driven form and need to implement the set of scan states of this mode.
- You need to define a new scan state for an existing scan mode of a barcode-driven from.

Scan State Classes

You create particular instances of the `ScanState<TScanState>` class in the `ScanMode<TScanBasis>.CreateStates()` method. The order of states used in this method doesn't imply the actual order of the input states.

A scan state class can be one of the following types:

- `EntityState<TScanBasis, TEntity>`: An input scan state that is used to transform a barcode to a particular entity, validate it, and then apply it to the barcode-driven form
- `ConfirmationState<TScanBasis>`: A confirmation scan state that validates and confirms all changes accumulated by other scan states and completes the barcode processing cycle
- `CommandOnlyState<TScanBasis>`: A built-in scan state that rejects any non-fixed input and reports to users that they should use only commands

Usually a scan mode has multiple input states and no more than one confirmation state.

The following diagram shows the relationships between these classes. For details about the methods and properties of these classes, see *ScanState<TScanBasis> Class*, *EntityState<TScanBasis,TEntity> Class*, *ConfirmationState<TScanBasis> Class*, and *CommandOnlyStateBase<TScanBasis> Class*.

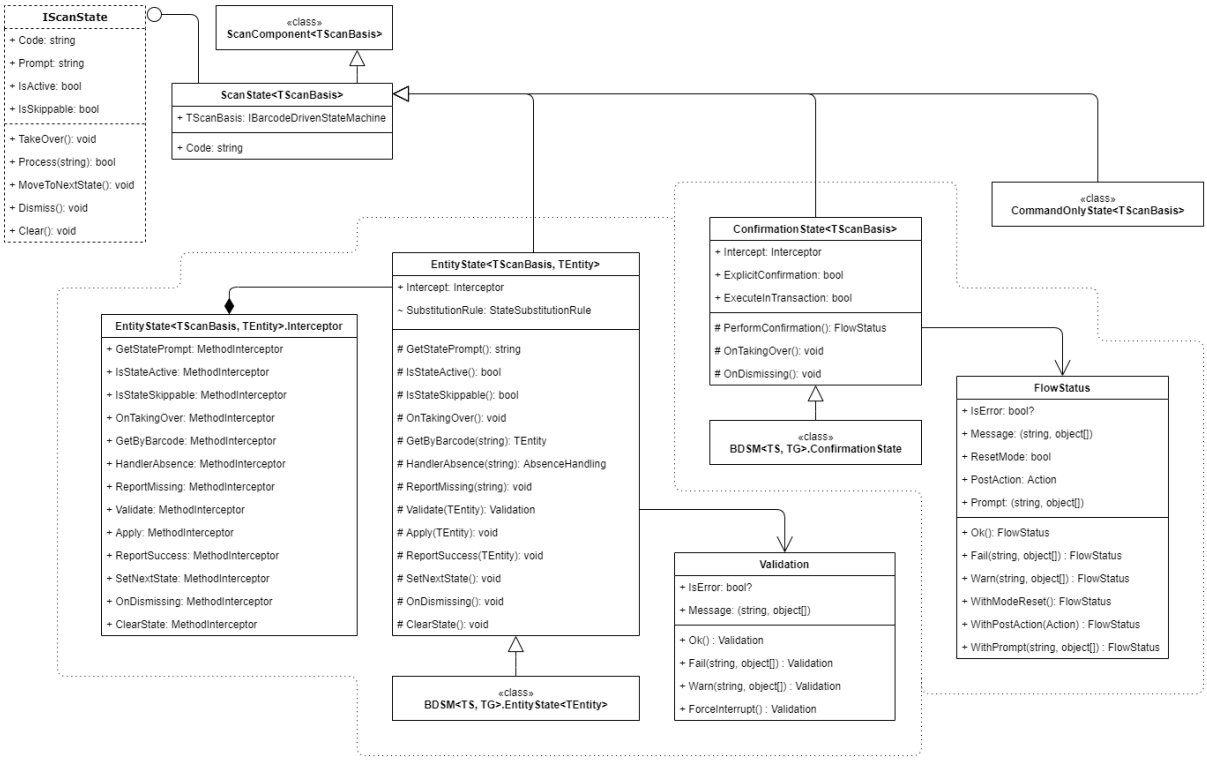


Figure: Scan state classes

Barcode Scan States: To Create the Set of Scan States

This activity will walk you through the creation of the set of barcode scan states.

Story

Suppose that you are implementing a scan mode for a custom barcode-driven form. The barcode scan class of this form uses the `WarehouseManagementSystem<TSelf,TGraph>` base scan class.

You need to define two new scan states for this scan mode: one input state and one confirmation state. Three other input states will be inherited from the base scan class.

Process Overview

You will create particular instances of the `ScanState<TScanState>` class in the `ScanMode<TScanBasis>.CreateStates()` method.

> You will define the classes for scan states in the following activities: *Barcode Scan States: To Create the Input State*, *Barcode Scan States: To Create the Confirmation State*, and *Extension of Scan Components: To Extend Predefined Scan States for a Custom Form*.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance as described in the *Test Instance: To Deploy an Instance* prerequisite activity.

2. Create a barcode scan class as described in the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Define the required properties of the scan mode as described in the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.

Step: Defining the Set of Scan States

To create the set of scan states, in the scan mode class, add the following code.

```
public sealed class CountMode : ScanMode
{
    ...

    protected override IEnumerable<ScanState<INScanCount>> CreateStates()
    {
        yield return new RefNbrState(); // new state introduced in the CountMode
        yield return new LocationState(); // state inherited from WMSBase
        yield return new InventoryItemState(); // state inherited from WMSBase
        yield return new LotSerialState(); // state inherited from WMSBase
        yield return new ConfirmState(); // new state introduced in the CountMode
    }

    ...
}
```

Barcode Scan States: Input State

The input state is a barcode scan state that is used to transform a barcode to a particular entity, validate it and then apply it to the barcode-driven form, without changing a certain document.

Required Properties

For an input state, you must define the following required properties:

- `Code`, which is the state identifier.
- `StatePrompt`, which is an input prompt. The input prompt should clearly explain what is expected from a user.

Cycle-Processing Methods

You can also implement the following additional methods, which affect the order in which the states are executed: `IsStateActive()` and `IsStateSkippable()`.

Depending on the purpose of the state, you may need to implement some of the methods shown in the following code.

```
protected override TEntity GetByBarcode(string barcode) { ... }
protected override AbsenceHandling.Of<TEntity> HandleAbsence(string barcode)  { ... }
protected override void ReportMissing(string barcode)  { ... }
protected override Validation Validate(TEntity entity) { ... }
protected override void Apply(TEntity entity) { ... }
protected override void ReportSuccess(TEntity entity)  { ... }
protected override void SetNextState() { ... }
```

```
protected override void ClearState() { ... }
```

In the `GetByBarcode()` method, you can define how a barcode should be transformed into the entity that it represents. We recommend that in this method you define the query that requests the entity only by the keys and that you move all restrictions to the `Validate()` method.

You can provide the validation of an entity in the `Validate()` method. If in this method implementation, you need to use the field's UI representation, you can retrieve it by using the `Basis.SightOf<>()` method.

You usually implement both the `Apply()` method and the `ClearState()` method. The `Apply()` method applies the found entity to the state of the barcode-driven form. The method is called when the barcode is processed by the state upon data input. The `ClearState()` method reverts the application done by the `Apply()` method. The `ClearState()` method is called once the `Basis.Clear<TState>()` or `ScanMode.Clear<TState>()` method is executed, which usually occurs when the mode is reset. For details on the resetting of the mode, see *Resetting a Barcode Scan Mode*.

For details about these methods, see *EntityState<TScanBasis,TEntity> Class*.

The following diagram illustrates the processing cycle.

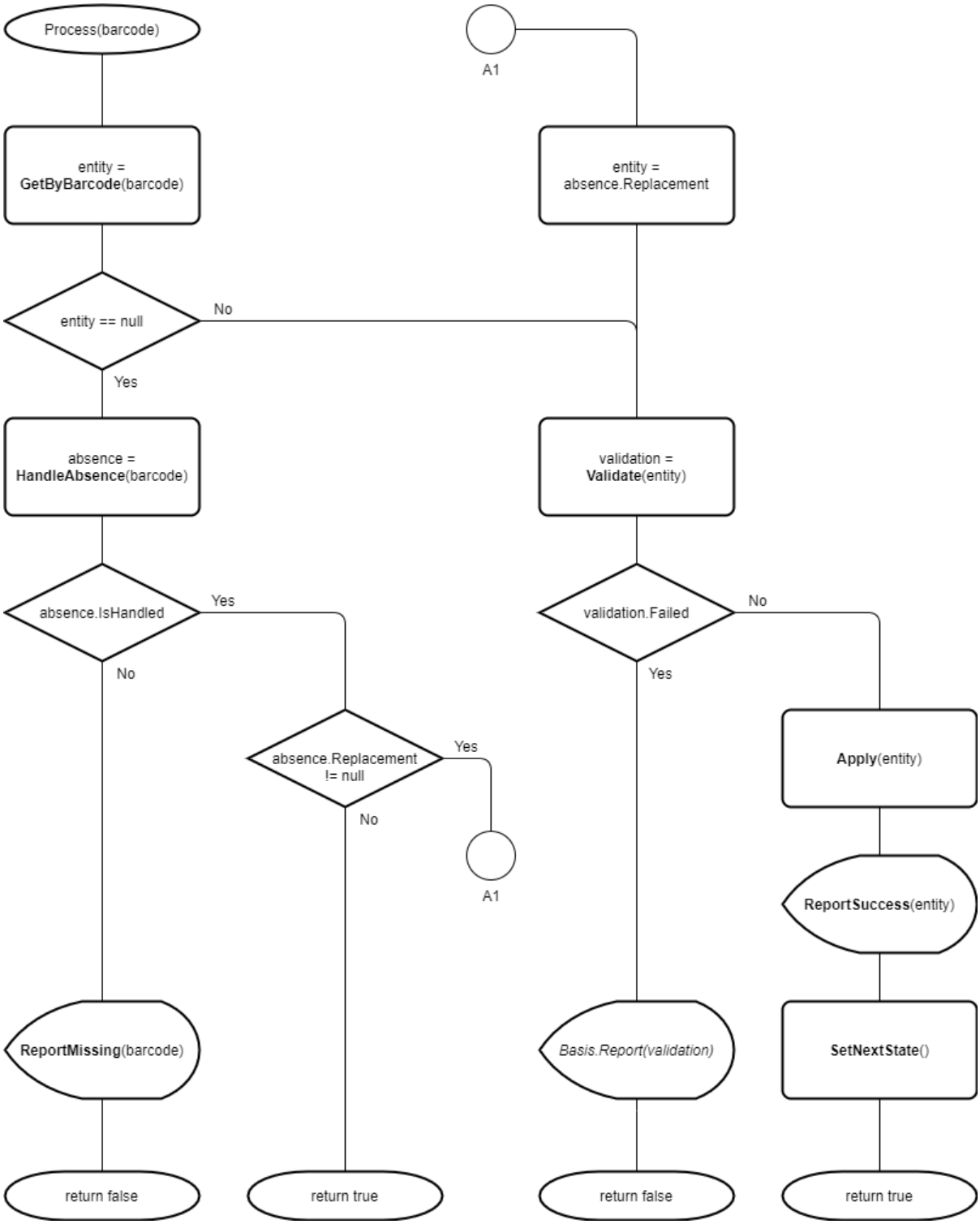


Figure: Input processing cycle

Barcode Scan States: To Create the Input State

This activity will walk you through the creation of the input scan state.

Story

Suppose that you are implementing a scan mode for a custom barcode-driven form. The barcode scan class of this form uses the `WarehouseManagementSystem<TSelf,TGraph>` base scan class.

For this scan mode, you need to define the input scan state, which handles the reference number of a document.

Process Overview

Because you need the input state to belong to a certain mode, you will nest the input state in the `CountMode` scan mode class.

The base barcode scan class already has a blank class for creating a state that handles the reference number of a document. Therefore, you can skip the implementation of particular properties and methods. You will not implement the following members:

- `Code`: It will use the default implementation of `WMSBase.RefNbrState<INPIHeader>`, which is *RNBR*.
- `IsStateActive()`: It will use the default implementation of the `EntityState<TBasis, TEntity>` class, which returns `true`.
- `IsStateSkippable()`: It will use the default implementation of the `WMSBase.RefNbrState<INPIHeader>` class, which is `"Basis.RefNbr != null && Basis.Header.ProcessingSucceeded != true"`.
- `SetNextState()`: It will use the default implementation, which moves the system to the next state based on the transition map.

In the `Validate()` method, you will reuse the method introduced in *Barcode Scan Class: To Create a Barcode Scan Class*.

System Preparation

Before you begin performing the steps of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Create the set of scan states by performing the *Barcode Scan States: To Create the Set of Scan States* prerequisite activity.

Step 1: Defining the Required Members

To create the input state, do the following:

1. In the `CountMode` class, create the `RefNbrState` class, as follows.

```
public class INScanCount : WMSBase
{
    ...
    public sealed class CountMode : ScanMode
    {
        ...
```

```
        public sealed class RefNbrState : WMSBase.RefNbrState<INPIHeader>
        {

        }
    }
}
```

2. In the `RefNbrState` class, define the required `StatePrompt` member, as follows.

```
 public sealed class RefNbrState : WMSBase.RefNbrState<INPIHeader>
 {
     protected override string StatePrompt => Msg.Prompt;

     [PXLocalizable]
     public abstract class Msg
     {
         public const string Prompt = "Scan a reference number of the PI count.";
     }
}
```

Step 2: Implementing the Cycle-Processing Members

Implement the cycle-processing members of the state as follows:

1. In the `RefNbrState` class, define how a barcode should be transformed into an entity it represents as follows.

```
public sealed class RefNbrState : WMSBase.RefNbrState<INPIHeader>
{
    ...
    protected override INPIHeader GetByBarcode(string barcode) =>
        INPIHeader.PK.Find(Basis, barcode);
    ...
}
```

2. Define the response that is displayed to a user if the state cannot find an entity, as follows.

```
public sealed class RefNbrState : WMSBase.RefNbrState<INPIHeader>
{
    ...
    protected override void ReportMissing(string barcode) =>
        Basis.ReportError(Msg.Missing, barcode);
    ...
    [PXLocalizable]
    public abstract class Msg
    {
        ...
        public const string Missing = "The {0} PI count was not found.";
    }
}
```

3. Provide the entity validation, as shown in the following code.

```
public sealed class RefNbrState : WMSBase.RefNbrState<INPIHeader>
{
    ...
    protected override Validation Validate(INPIHeader entity) =>
```

```
        Basis.IsDocumentStatusEditable(entity.Status)
        ? Validation.Ok
        : Validation.Fail(Msg.InvalidStatus,
            Basis.SightOf<INPIHeader.status>(entity));
    ...
    [PXLocalizable]
    public abstract class Msg
    {
        ...
        public const string InvalidStatus =
            "Document has the {0} status, cannot be used for count.";
    }
}
```

4. Implement the `Apply()` and `ClearState()` methods as follows.

```
public sealed class RefNbrState : WMSBase.RefNbrState<INPIHeader>
{
    ...
    protected override void Apply(INPIHeader entity)
    {
        Basis.RefNbr = entity.PIID;
        Basis.SiteID = entity.SiteID;
        Basis.NoteID = entity.NoteID;
        Basis.DocumentView.Current = entity;
    }
    protected override void ClearState()
    {
        Basis.RefNbr = null;
        Basis.SiteID = null;
        Basis.NoteID = null;
        Basis.DocumentView.Current = null;
    }
    ...
}
```

5. Define the response that is displayed to the user if the entity was successfully found, validated, and applied to the state of the barcode-driven form, as follows.

```
public sealed class RefNbrState : WMSBase.RefNbrState<INPIHeader>
{
    ...
    protected override void ReportSuccess(INPIHeader entity) =>
        Basis.ReportInfo(Msg.Ready, entity.PIID);
    ...
    [PXLocalizable]
    public abstract class Msg
    {
        ...
        public const string Ready =
            "The {0} PI count has been loaded and is ready for processing.";
    }
}
```

Barcode Scan States: Confirmation State

The confirmation state is a special scan state that validates and confirms all the data accumulated by input states and completes the barcode processing cycle by applying changes based on this data to a certain document. There are two types of confirmation strategies: explicit and implicit. An explicit confirmation strategy shows the prompt to confirm an action. An implicit confirmation strategy automatically executes the confirmation process.

Required Properties

For a confirmation state, you must implement the `Prompt` required property. You use the property to specify the prompt that is shown to a user to confirm an action. In the `Prompt` property, you can use the `Basis.Localize()` method if you need to insert message parameters. If you do not need to use parameters in the message, you return the message itself. The system localizes it automatically.

Implementation of the Confirmation Process

The `PerformConfirmation()` method is an entry point for the confirmation process. The `FlowStatus` object is the result of the `PerformConfirmation()` method. This object indicates how the system should complete the confirmation. The `FlowStatus` class has multiple factory methods that facilitate proper object creation. For details about these methods, see *ConfirmationState<TScanBasis> Class*.

A confirmation process can contain complex logic, which should be split into multiple methods that can be overridden separately. While scan components should not introduce and use virtual methods, this does not mean that all the confirmation logic should be implemented in a single method. To implement this complex logic, you can use the `ScanExtension` components. The `ScanExtension` component is a `PXGraphExtension` that can be created and used in the scope of the barcode scan class and of all types of its components. Any scan component can delegate its responsibility to a particular `ScanExtension` component. Because the `ScanExtension` component is a `PXGraphExtension`, it can be customized as other graph extensions are customized; that is, it can be split into multiple virtual methods that can be overridden.

The component that delegates a part of its logic to a `ScanExtension` is called a complex component. You can learn how to customize complex scan components in *Extension of Scan Components: General Information*.

Change of the Confirmation Strategy

By default, the system uses the implicit confirmation strategy. However, you can change the preferred strategy by overriding the `ExplicitConfirmation` property of the barcode scan class. The following code shows an example of the approach that is used in the `INScanCount` class.

```
public class INScanCount : WMSBase
{
    ...

    public
        PXSetupOptional<INScanSetup,
        Where<INScanSetup.branchID.IsEqual<AccessInfo.branchID.FromCurrent>>>
        Setup;

    public override bool ExplicitConfirmation =>
        Setup.Current.ExplicitLineConfirmation == true;

    ...
}
```

Barcode Scan States: To Create the Confirmation State

This activity will walk you through the creation of the confirmation scan state.

Story

Suppose that you are implementing a scan mode for a custom barcode-driven form. The barcode scan class of this form uses the `WarehouseManagementSystem<TSelf,TGraph>` base scan class.

For this scan mode, you need to define the confirmation scan state. You need to split the confirmation logic into the following `virtual` methods:

- `Confirm()`: The entry point method of the scan extension component
- `CanConfirm()`: A method that is used to extend the validation logic
- `ConfirmRow()`: The method that contains the main logic of confirmation
- `FindDetailRow()`: The helper method that can be altered

Process Overview

You will create a `WMSBase.ConfirmationState` class descendant. In this class, you will define a `ScanExtension` class that will implement the confirmation logic.

In the `PerformConfirmation()` method, you will then get the `ScanExtension` object by using the `Get<TExtension>()` method and call its entry point method.

System Preparation

Before you begin performing the steps of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Create the set of scan states by performing the *Barcode Scan States: To Create the Set of Scan States* prerequisite activity.

Step 1: Defining the Required Member

To create the confirmation state, do the following:

1. In the `CountMode` class, create the `ConfirmState` class as follows.

```
public class INScanCount : WMSBase
{
    ...
    public sealed class CountMode : ScanMode
    {
        ...

        public sealed class ConfirmState : WMSBase.ConfirmationState
```

```
        {

        }
    }
}
```

2. Define the required member as follows.

```
public sealed class ConfirmState : WMSBase.ConfirmationState
{
    public override string Prompt =>
        Basis.Localize(Msg.Prompt, Basis.SightOf<WMSScanHeader.inventoryID>(),
            Basis.Qty, Basis.UOM);

    [PXLocalizable]
    new public abstract class Msg
    {
        public const string Prompt = "Confirm counting {0} x {1} {2}.";
    }
}
```

Step 2: Implementing the Confirmation Process

Implement the confirmation process as follows:

1. Define a scan state-related extension class inside the scan state and place in the extension class only component related logic, as shown in the following code.

```
public sealed class ConfirmState : WMSBase.ConfirmationState
{
    ...
    public class Logic : ScanExtension
    {
        public virtual FlowStatus Confirm()
  {
      if (!CanConfirm(out var error))
     return error;
                return ConfirmRow();
  }

        protected virtual bool CanConfirm(out FlowStatus error)
        {
            if (Basis.Document == null)
            {
                error = FlowStatus.Fail(RefNbrState.Msg.NotSet);
                return false;
            }

            if (Basis.DocumentIsEditable == false)
            {
                error = FlowStatus.Fail(RefNbrState.Msg.InvalidStatus,

 Basis.DocumentView.Cache.GetStateExt<INPIHeader.status>(Basis.Document));
                return false;
            }

            if (Basis.InventoryID == null)
```

```
            {
                error = FlowStatus.Fail(InventoryItemState.Msg.NotSet);
                return false;
            }

            if (Basis.CurrentMode.HasActive<LotSerialState>() && Basis.LotSerialNbr ==
null)
            {
                error = FlowStatus.Fail(LotSerialState.Msg.NotSet);
                return false;
            }

            if (Basis.CurrentMode.HasActive<LotSerialState>() &&
                Basis.SelectedLotSerialClass.LotSerTrack ==
INLotSerTrack.SerialNumbered &&
                Basis.BaseQty != 1)
            {
                error =
FlowStatus.Fail(InventoryItemState.Msg.SerialItemNotComplexQty);
                return false;
            }

            error = FlowStatus.Ok;
            return true;
        }

        protected virtual FlowStatus ConfirmRow()
        {
            INPIDetail row = FindDetailRow();

            decimal? newQty = row?.PhysicalQty ?? 0;
            if (Basis.Remove == true)
                newQty -= Basis.BaseQty;
            else
                newQty += Basis.BaseQty;

            if (Basis.CurrentMode.HasActive<LotSerialState>() &&
                Basis.SelectedLotSerialClass.LotSerTrack ==
INLotSerTrack.SerialNumbered &&
                newQty.IsNotIn(0, 1))
            {
                return
FlowStatus.Fail(InventoryItemState.Msg.SerialItemNotComplexQty);
            }

            if (row == null)
            {
                row = (INPIDetail)Basis.Details.Cache.CreateInstance();
                row.PIID = Basis.RefNbr;
                row.LineNbr =
(int)PXLineNbrAttribute.NewLineNbr<INPIDetail.lineNbr>(Basis.Details.Cache,
Basis.Document);
                row.InventoryID = Basis.InventoryID;
                row.SubItemID = Basis.SubItemID;
                row.SiteID = Basis.SiteID;
                row.LocationID = Basis.LocationID;
                row.LotSerialNbr = Basis.LotSerialNbr;
```

```
                row.PhysicalQty = Basis.BaseQty;
                row.BookQty = 0;
                row = Basis.Details.Insert(row);

                Basis.SaveChanges();

                row = Basis.Details.Locate(row) ?? row;
            }

            Basis.Details.SetValueExt<INPIDetail.physicalQty>(row, newQty);
            row = Basis.Details.Update(row);
            Basis.SaveChanges();

            Basis.DispatchNext(
                null,
                Basis.SightOf<WMSScanHeader.inventoryID>(), Basis.Qty, Basis.UOM);

            return FlowStatus.Ok;
        }

        protected virtual INPIDetail FindDetailRow()
        {
            var findDetailCmd =
 BqlCommand.CreateInstance(Basis.Details.View.BqlSelect.GetType());

            findDetailCmd = findDetailCmd.WhereAnd<Where<
                INPIDetail.inventoryID.IsEqual<WMSScanHeader.inventoryID.FromCurrent>.
                And<INPIDetail.siteID.IsEqual<WMSScanHeader.siteID.FromCurrent>>
            >>();

            if (Basis.CurrentMode.HasActive<LocationState>() && Basis.LocationID !=
 null)
                findDetailCmd =
 findDetailCmd.WhereAnd<Where<INPIDetail.locationID.IsEqual<
                    WMSScanHeader.locationID.FromCurrent>>>();

            if (Basis.CurrentMode.HasActive<LotSerialState>() && Basis.LotSerialNbr !
= null)
                findDetailCmd =
 findDetailCmd.WhereAnd<Where<INPIDetail.lotSerialNbr.IsEqual<
                    WMSScanHeader.lotSerialNbr.FromCurrent>>>();

            var findDetailView = Basis.Graph.TypedViews.GetView(findDetailCmd, false);
            var findResultSet = (PXResult<INPIDetail>)findDetailView.SelectSingle();

            return findResultSet;
        }
    }

    ...
}
```

2. Implement the `PerformConfirmation()` method as follows.

```
public sealed class ConfirmState : WMSBase.ConfirmationState
{
    ...
```

```
    protected override FlowStatus PerformConfirmation() =>
        Get<Logic>().Confirm();
    ...
}
```

3. Fix or suppress the *PX1016* error that is displayed by Acuminator for the `Logic` class.

Implementing Barcode Scan Transitions

In this chapter, you will learn about barcode scan transitions, which you can use to define the rules of transitions between scan states.

Barcode Scan Transitions: General Information

`ScanTransition<TScanBasis>` is a class for a barcode scan transition. This class represents a component that defines the rules of automatic transitions between scan states.

Learning Objectives

In this chapter, you will learn how to do the following:

- Implement transitions between scan states
- Implement a simple transition flow between scan states

Applicable Scenarios

You implement scan transitions in the following cases:

- You have defined a set of scan states of a custom scan mode. You need to implement transitions between these states.
- You have defined a new scan state of a predefined scan mode. You need to implement transitions between the predefined scan states and the new scan state.

Transition Between Scan States

You move the barcode-driven engine to another scan state in either of the following ways:

- Call `Basis.SetScanState(string state)` or `Basis.SetScanState<TScanState>()` to change the current scan state to another certain scan state. This way requires a particular state to be specified and is not flexible because scan states become highly coupled.
- Call `Basis.DispatchNext()` to indicate to the system that the current scan state should be moved further. This way does not include information about a specific next scan state. The system derives the information about the next scan state from the transition map, which is defined by the scan transition component.

Transition Map

You can use the scan transition component to create a complex map of transitions by using conditions and additional actions that can be performed when the transition is triggered.

Unlike all other scan components, `ScanTransition<TScanBasis>` is not an `abstract` class. However, it still can be configured by a condition or by additional transition logic. Also, in some cases, a transition map does not contain any branches; it contains only a group of optional states. In these cases, the creation of the transition map can be simplified by the `ScanStateFlow<TScanBasis>` class, which you can use to create the transition map sequentially.

The following diagram shows the classes and interfaces related to scan transitions.

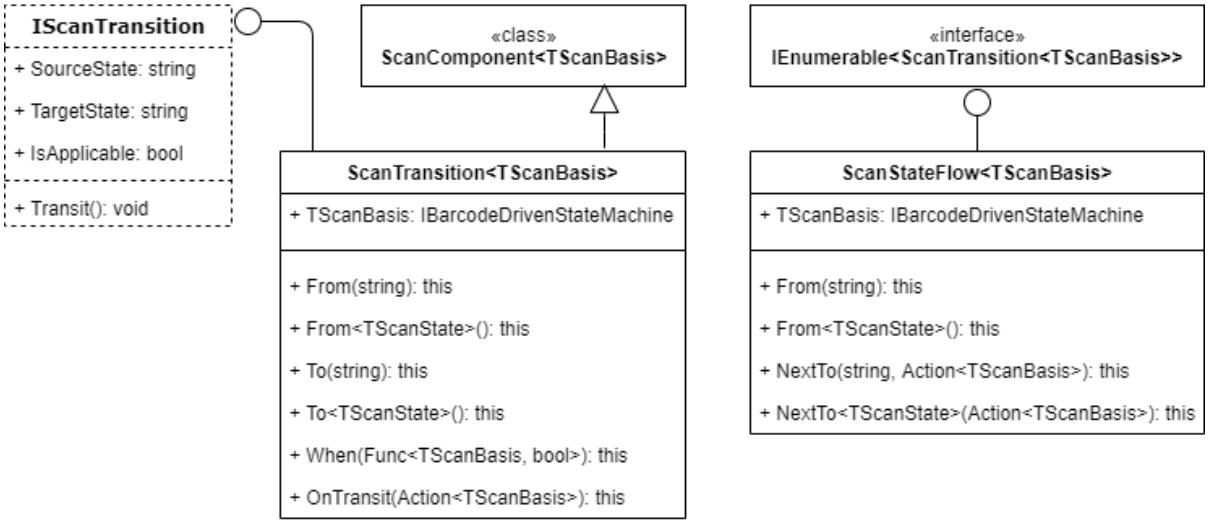


Figure: Scan transition classes and interfaces

Implementation of the Transition Map

You create particular instances of the `ScanTransition<TScanBasis>` class in the `ScanMode<TScanBasis>.CreateTransitions()` method.

To define the transition map, you can use the following methods:

- `Transition(Func<ScanTransition<TSelf>, ScanTransition<TSelf>> config)`

  The `Transition` object can be used for producing a branching structure of transitions. This approach is shown in *Barcode Scan Transitions: To Implement Transitions*.
- `StateFlow(Func<ScanStateFlow<TSelf>.IFrom, ScanStateFlow<TSelf>.IFlow> config)`

  The `StateFlow` object implements the `IEnumerable<ScanTransition<TScanBasis>>` interface—that is, it produces a sequence of transition objects. The `StateFlow` object cannot be used for producing a branching structure of transitions. However, it supports additional actions that can be performed upon transition. To specify this actions, you use the overload of the `NextTo` method that accepts an `Action<TScanBasis>` delegate. This approach is shown in *Barcode Scan Transitions: To Implement a Simple Transition Flow*.

Barcode Scan Transitions: To Implement Transitions

This activity will walk you through the implementation of scan transitions. This activity shows the general approach that can be used for producing a branching structure of transitions.

Story

Suppose that you have defined the following scan states for the scan mode of a custom form:

- `RefNbrState`
- `LocationState`
- `InventoryItemState`
- `LotSerialState`
- `ConfirmState`

You need to implement transition rules between these states as follows:

1. From `RefNbrState` to `LocationState`
2. From `LocationState` to `InventoryItemState`
3. From `InventoryItemState` to `LotSerialState`

> You will not implement the last transition, which is from `LotSerialState` to `ConfirmState`, because it is performed automatically once the system detects that there is a confirmation state and that the last input state is trying to move the barcode-driven engine further.

Process Overview

You will use the `ScanMode.Transition()` helper method to implement the transitions.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Create the set of scan states by performing the *Barcode Scan States: To Create the Set of Scan States* prerequisite activity.

Step: Defining Transition Rules

Define scan transition rules as follows.

```
public sealed class CountMode : ScanMode
{
    ...

    protected override IEnumerable<ScanTransition<INScanCount>> CreateTransitions()
    {
        yield return Transition(t => t.From<RefNbrState>().To<LocationState>());
        yield return Transition(t => t.From<LocationState>().To<InventoryItemState>());
        yield return Transition(t => t.From<InventoryItemState>().To<LotSerialState>());
    }

    ...
}
```

Barcode Scan Transitions: To Implement a Simple Transition Flow

This activity will walk you through the implementation of a simple transition flow, which cannot be used for producing a branching structure of transitions.

Story

Suppose that you have defined the following scan states for the scan mode of a custom form:

- `RefNbrState`
- `LocationState`
- `InventoryItemState`
- `LotSerialState`
- `ConfirmState`

You need to implement transition rules between these states as follows:

1. From `RefNbrState` to `LocationState`
2. From `LocationState` to `InventoryItemState`
3. From `InventoryItemState` to `LotSerialState`

> You will not implement the last transition, which is from `LotSerialState` to `ConfirmState`, because it is performed automatically once the system detects that there is a confirmation state and that the last input state is trying to move the barcode-driven engine further.

Process Overview

You will use the `ScanMode.StateFlow()` helper method to implement the simple one-by-one chain of input states.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Create the set of scan states by performing the *Barcode Scan States: To Create the Set of Scan States* prerequisite activity.

Step: Defining the Transition Flow

Define the scan transition flow as follows.

```
public sealed class CountMode : ScanMode
{
    ...

    protected override IEnumerable<ScanTransition<INScanCount>> CreateTransitions()
    {
        return StateFlow(flow => flow
            .From<RefNbrState>()
            .NextTo<LocationState>()
            .NextTo<InventoryItemState>()
```

```
            .NextTo<LotSerialState>());
    }

    ...
}
```

Defining the List of Barcode Scan Commands

In this chapter, you will learn about the commands that can be implemented on an Acumatica ERP barcode-driven form. You will also learn how to define the list of commands and a custom command.

Barcode Scan Commands: General Information

A barcode scan command is a piece of logic that can be executed in any scan state either by clicking a button associated with this command in the UI, or by scanning a fixed barcode associated with it. For example, this logic can change the current state of the barcode-driven engine or start a long-running process.

Learning Objectives

In this chapter, you will learn how to do the following:

- Define the set of barcode scan commands of a scan mode
- Implement a custom barcode scan command
- Add to a scan mode the logic that handles an input of the quantity

Applicable Scenarios

You define scan commands in the following cases:

- You have created a scan mode on a custom form and need to implement the set of scan commands of this mode.
- You have added a new scan mode on an existing barcode-driven form and need to implement the set of scan commands of this mode.
- You have created a scan mode on a custom form and need to add to this scan mode the command for the input of the quantity

Scan Command Component

The logic of a scan command is implemented in the `ScanCommand` component. You use `ScanCommand<TScanBasis>` to bind custom logic to a particular barcode command.

All commands produce a related `PXAction`. Therefore, the command can be executed in two ways: by clicking the button in the user interface, or by scanning a barcode.

The following diagram shows the classes and interfaces that are related to the scan command component. For details about the properties and methods of these classes and interfaces, see *ScanCommand<TScanBasis> Class*.

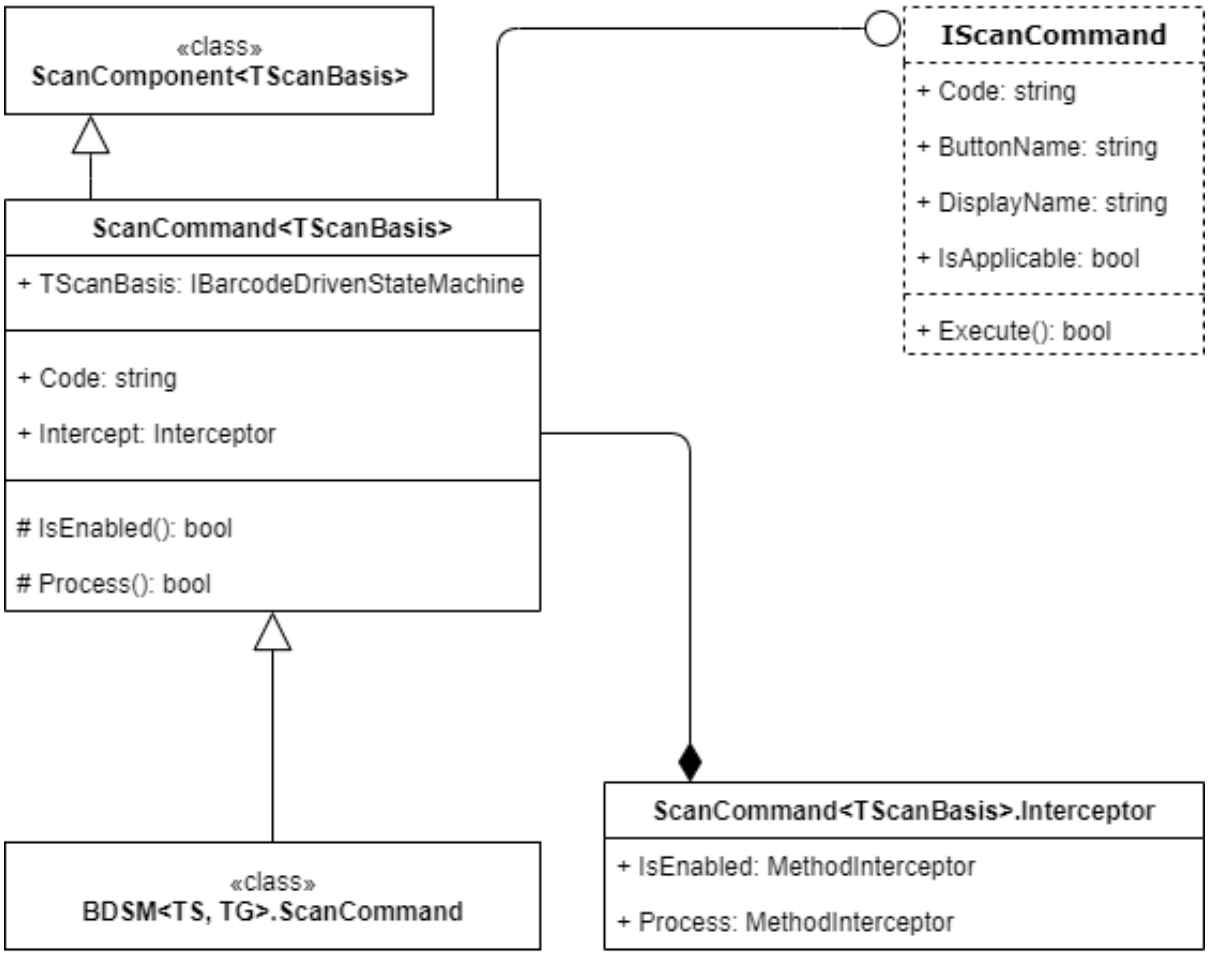


Figure: Scan command classes and interfaces

Implementation of a Scan Command

You create particular instances of this class in the `ScanMode<TScanBasis>.CreateCommands()` method.

For each scan command, you specify the following required properties:

- `Code`: The code that is used to perform the command. The system automatically adds the `*` character to the beginning of all command codes and uses this character to distinguish them from the codes intended for processing by input state processors.
- `ButtonName`: The name of the `PXAction` instance that is created for this command component.
- `DisplayName`: The display name of the button for the `PXAction` instance that is created for this command component.
- `IsEnabled`: A Boolean value that indicates (if set to `true`) that the command can be executed.

In the `Process()` method, you implement the logic that is performed when the scan command is executed.

If you want to run a long-running process in the scope of a command component, you use the `Basis.WaitFor()` method.

Barcode Scan Commands: To Define the List of Commands

This activity will walk you through the process of defining the list of commands that are available in a scan mode.

Story

Suppose that you are creating a custom scan mode for a barcode-driven form. You need to define two new scan commands for this scan mode. One command, `RemoveCommand`, will be inherited from the base barcode scan class, which is the `WMSBase` class. The other, `ConfirmCommand`, will be introduced in this scan mode. In this activity, you will define the list of commands.

> You will define the `ConfirmCommand` class in *Barcode Scan Commands: To Implement a Custom Command*.

Process Overview

You will create particular instances of scan commands in the `ScanMode<TScanBasis>.CreateCommands()` method.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.

Step: Defining the List of Commands

To define the list of commands, in the scan mode class, add the following code.

```
public sealed class CountMode : ScanMode
{
    ...

    protected override IEnumerable<ScanCommand<INScanCount>> CreateCommands()
    {
        yield return new RemoveCommand();
        yield return new ConfirmCommand();
    }

    ...
}
```

Barcode Scan Commands: To Implement a Custom Command

This activity will walk you through the process of defining a custom command for a scan mode.

Story

Suppose that you are creating a custom scan mode for a barcode-driven form. You have defined the list of commands for this scan mode and introduced the `ConfirmCommand` command in this scan mode. You now need to implement the custom `ConfirmCommand` command.

Process Overview

You will specify the required properties of the `ConfirmCommand` scan command, which is a new custom command, and implement its `Process()` method.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Define the list of commands for the scan mode by performing the *Barcode Scan Commands: To Define the List of Commands* prerequisite activity.

Step: Defining the Custom Command

To implement the `ConfirmCommand`, do the following:

1. In the `CountMode` class, add the `ConfirmCommand` class for the custom command and define its required properties, as shown in the following code.

```
public class INScanCount : WMSBase
{
    public sealed class CountMode : ScanMode
    {
        ...

        public sealed class ConfirmCommand : ScanCommand
        {
            public override string Code => "CONFIRM";
            public override string ButtonName => "scanConfirmDocument";
            public override string DisplayName => Msg.DisplayName;
            protected override bool IsEnabled => Basis.DocumentIsEditable;

            [PXLocalizable]
            public abstract class Msg
            {
                public const string DisplayName = "Confirm";
            }
        }

        ...
    }
```

```
}
```

2. In the `RefNbrState.Msg` class, add the following message.

```
public const string NotSet = "The document number is not selected.";
```

3. In the `CountMode` class, add the `Process()` method implementation, as shown in the following code.

```
public class INScanCount : WMSBase
{
    public sealed class CountMode : ScanMode
    {
        ...

        public sealed class ConfirmCommand : ScanCommand
        {
            ...

            protected override bool Process()
            {
                if (Basis.Document == null)
                    Basis.ReportError(RefNbrState.Msg.NotSet);
                if (Basis.DocumentIsEditable == false)
                    Basis.ReportError(RefNbrState.Msg.InvalidStatus,
                        Basis.DocumentView.Cache.GetStateExt<INPIHeader.status>(
                        Basis.Document));

                BqlCommand nullPhysicalQtyCmd = BqlCommand.CreateInstance(
                    Basis.Details.View.BqlSelect.GetType());
                nullPhysicalQtyCmd = nullPhysicalQtyCmd.WhereAnd<Where<
                    INPIDetail.physicalQty.IsNull>>();
                PXView nullPhysicalQtyView = Basis.Graph.TypedViews.GetView(
                    nullPhysicalQtyCmd, false);

                foreach (INPIDetail detail in nullPhysicalQtyView.SelectMulti().
                    RowCast<INPIDetail>())
                {
                    Basis.Details.SetValueExt<INPIDetail.physicalQty>(detail, 0m);
                    Basis.Details.Update(detail);
                }

                Basis.SaveChanges();

                Basis.DocumentView.Current = null;
                Basis.CurrentMode.Reset(fullReset: true);
                Basis.ReportInfo(Msg.CountConfirmed);

                return true;
            }

            [PXLocalizable]
            public abstract class Msg
            {
                ...
                public const string CountConfirmed = "The count has been saved.";
            }
        }
```

```
        ...
    }
}
```

Barcode Scan Commands: To Add Quantity Support

This activity will walk you through adding the logic that handles input of a quantity.

Story

Suppose that you are creating a custom scan mode for a barcode-driven form. You need to add the logic that handles the input of a quantity in this scan mode. You will use the `QtySupport` extension defined in the `WarehouseManagementSystem<TSelf,TGraph>` class. The logic of the `QtySupport` extension cannot be used until you create an empty descendant for this extension class.

Process Overview

In this activity, you will do the following:

1. Define an empty descendant of the `QtySupport` extension.
2. Override the `UseQtyCorrection` method of the barcode scan class.
3. Add the `QtySupport.SetQtyCommand` component in the `CreateCommands` method.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Define the list of commands for the scan mode by performing the *Barcode Scan Commands: To Define the List of Commands* prerequisite activity.

Step: Adding the Quantity Support Extension

To add the quantity support extension, do the following:

1. In the `INScanCount` class, define an empty descendant of the `QtySupport` extension, as shown in the following code.

```
public class INScanCount : WMSBase
{
    ...
    public new class QtySupport : WMSBase.QtySupport { }
    ...
}
```

2. Fix or suppress the *PX1016* error that is displayed by Acuminator for the class.

3. In the `INScanCount` class, define the `Setup` data view, as shown in the following code.

```
public class INScanCount : WMSBase
{
    ...
    public PXSetupOptional<INScanSetup,
        Where<INScanSetup.branchID.IsEqual<
            AccessInfo.branchID.FromCurrent>>> Setup;
    ...
}
```

4. Override the `UseQtyCorrection` method, as shown below.

```
public class INScanCount : WMSBase
{
    ...
    protected override bool UseQtyCorrection =>
        Setup.Current.UseDefaultQtyInCount != true;
    ...
}
```

5. Add the `QtySupport.SetQtyCommand` component to the command list as follows.

```
public sealed class CountMode : ScanMode
{
    ...
    protected override IEnumerable<ScanCommand<INScanCount>> CreateCommands()
    {
        yield return new RemoveCommand();
        yield return new QtySupport.SetQtyCommand();
        yield return new ConfirmCommand();
    }
    ...
}
```

Defining the Set of Scan Mode Redirects

In this chapter, you will learn about scan mode redirects, which you can use to change the mode of a barcode-driven form. You will also learn how to define the set of scan mode redirects.

Scan Mode Redirects: General Information

You can use the `ScanRedirect` component to give a user the ability to change the scan mode. A user will be able to change the scan mode in any scan state either by clicking a button associated with this redirect in the UI (only in the mobile app), or by scanning a barcode associated with it (only in the web application).

Learning Objectives

In this chapter, you will learn how to define the set of mode redirects.

Applicable Scenarios

You perform this business process in the following cases:

- You have created a scan mode on a custom form and need to implement the redirects for this mode.
- You have added a new scan mode on an existing barcode-driven form and need to implement the redirects for this mode.

The ScanRedirect Component

`ScanRedirect<TScanBasis>` is a component that contains logic related to a transition from one mode to another. It works similarly to the `ScanCommand<TScanBasis>` class. It produces a `PXAction` member and can be invoked by scanning a barcode.

The following diagram shows the classes and interfaces that are related to the scan redirect component. For details about the properties and methods of these classes and interfaces, see *ScanRedirect<TScanBasis> Class*.

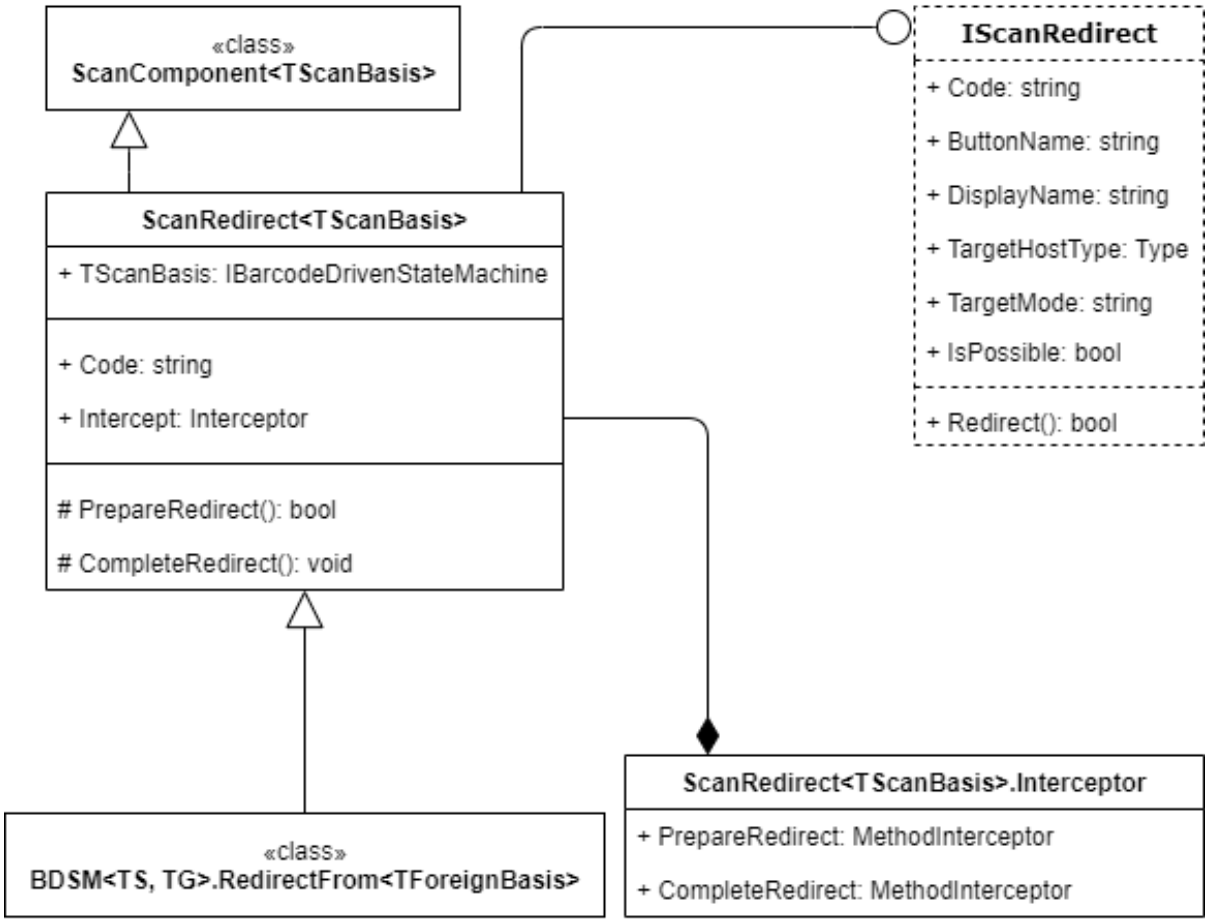


Figure: ScanRedirect component

Default Set of Mode Redirects

Because all predefined barcode-driven forms are known when the form is designed, the system provides a global list of redirects, which contains redirects to any barcode-driven form. You can access this list through the `AllWMSRedirects.CreateFor<TScanBasis>()` method. Therefore, you can implement redirection of a user from any predefined barcode-driven form to any other barcode-driven form.

If you need to add a redirect to a custom mode, you cannot extend this list itself, but you can create a generic extension that adds a redirect (to a new form) to all other forms.

Implementation of a Redirect Object

You create particular instances of the `ScanRedirect<TScanBasis>` class in the `ScanMode<TScanBasis>.CreateRedirects()` method. Typically, you use `AllWMSRedirects.CreateFor<TScanBasis>()` in this method to define the full set of redirects for a mode. However, the list of available redirects could still be listed manually if you create the proper redirect object.

No redirect object is used by its nesting class. Instead, other barcode-driven forms or modes use these objects. For a scan redirect object of a custom mode, you need to define the following required properties:

- `Code`: The code that is used to perform the redirect. The `@` character is added to the beginning of all redirect codes so that the system can distinguish them from the codes intended for processing by input state processors.
- `DisplayName`: The display name of the button for the `PXAction` instance that is created for this redirect.

- `IsPossible`: The Boolean value that specifies (if set to `true`) that it is possible to scan the redirect code and the redirect button is displayed in the UI. Redirects that are not possible are invisible in the UI.

You should keep in mind that any redirect object refers to two barcode-driven forms: the source and the target. Sometimes the source and the target can be the same form.

Scan Mode Redirects: To Define the List of Redirects

This activity will walk you through the process of defining the list of mode redirects.

Story

Suppose that you are creating a custom scan mode for a barcode-driven form. You need to define the default set of redirects for the form. You also need to define the redirect class for the custom mode so that redirects to this mode can be performed from other barcode-driven forms.

Process Overview

You will use the `AllWMSRedirects.CreateFor<TScanBasis>()` method to specify the default set of mode redirects for the form.

You will also define the redirect class. In this class, you will specify the `Code` property and the `DisplayName` property.

System Preparation

Before you begin performing the steps of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.

Step 1: Defining the Default Set of Mode Redirects

To define the default list of redirects, in the `CountMode` class, add the following code.

```
public sealed class CountMode : ScanMode
{
    ...

    protected override IEnumerable<ScanRedirect<INScanCount>> CreateRedirects()
    {
        return AllWMSRedirects.CreateFor<INScanCount>();
    }

    ...
}
```

Step 2: Adding the Redirect Class for the Mode

Add the redirect class as follows:

1. In the `INScanCount` class, define the redirect class, as shown in the following code.

```
public class INScanCount : WMSBase
{
    ...

    public new sealed class RedirectFrom<TForeignBasis> :
        WMSBase.RedirectFrom<TForeignBasis>
        where TForeignBasis : PXGraphExtension, IBarcodeDrivenStateMachine
    {
    }
}
```

2. Define the localizable message for the name of the mode redirect, as shown in the following code.

```
public new sealed class RedirectFrom<TForeignBasis> :
    WMSBase.RedirectFrom<TForeignBasis>
    where TForeignBasis : PXGraphExtension, IBarcodeDrivenStateMachine
{
    [PXLocalizable]
    public abstract class Msg
    {
        public const string DisplayName = "PI Count";
    }
}
```

3. Define the redirect identifier and the display name of the button that is generated for this redirect, as shown below.

```
public class INScanCount : WMSBase
{
    ...

    public new sealed class RedirectFrom<TForeignBasis> :
        WMSBase.RedirectFrom<TForeignBasis>
        where TForeignBasis : PXGraphExtension, IBarcodeDrivenStateMachine
    {
        public override string Code => "COUNT";
        public override string DisplayName => Msg.DisplayName;

        ...
    }
}
```

4. Specify whether it is possible to scan the redirect code as follows.

```
public new sealed class RedirectFrom<TForeignBasis> :
    WMSBase.RedirectFrom<TForeignBasis>
    where TForeignBasis : PXGraphExtension, IBarcodeDrivenStateMachine
{
    ...
    public override bool IsPossible =>
        PXAccess.FeatureInstalled<PX.Objects.CS.FeaturesSet.wMSInventory>();
```

```
    ...
}
```

Resetting a Barcode Scan Mode

In this chapter, you will learn about the resetting of a barcode scan mode. You need to define how the mode is cleared.

Reset of Barcode Scan Mode: General Information

You need to define how each barcode scan mode is cleared.

Learning Objectives

In this chapter, you will learn how to implement the resetting logic of the barcode scan mode.

Applicable Scenarios

You need to define the resetting logic for each barcode scan mode that you implement.

Resetting of the Scan Mode

The scan mode is reset as follows:

- A soft reset is executed once the scanning cycle is completed—that is, when a confirmation state reflects success. The system executes the `ScanMode.Reset(fullReset: false)` method for the soft reset.
- A hard reset is executed in the following cases:
  - If the scan mode is changed to another one.
  - If a confirmation state reports failure and requests a reset.
  - If a user executes the reset command either by clicking the **Reset** button of the form toolbar or by scanning the `*RESET` barcode. However, in this case, the `Basis.IsWithinReset` flag is set. This flag can be used in the mode's resetting logic to determine whether a particular state should be cleared or not.

  The system executes the `ScanMode.Reset(fullReset: true)` method for the hard reset.
- The whole form is reset once a user clicks the **Cancel** button on the form toolbar. This is the default behavior of an Acumatica ERP form. In this case, all screen caches are cleaned, which causes form reinitialization.

The `ScanMode.Reset()` method can be called from any other scan component either in soft mode or in hard mode.

> ⚠ You should not call the protected `ScanMode.ResetMode()` method within any component. This method is for override only. Moreover, the method can be intercepted. Therefore, in order to cause all interceptors to be applied, this method should be called indirectly via the `public ScanMode.Reset()` method.

Reset of Barcode Scan Mode: To Define the Resetting Logic

This activity will walk you through the resetting of the scan mode logic.

Story

Suppose that you are creating a custom barcode-driven form. You have defined a barcode scan class for this form and implemented the scan mode, including its required properties and its states, which are `LotSerialState`, `InventoryItemState`, `LocationState`, and `RefNbrState`. Now you need to define how the scan mode is cleared.

You want `LotSerialState` and `InventoryItemState` to be cleared with a soft reset every time the cycle ends because neither of these states should store its data in memory between cycles. (Each cycle demands the entry of the inventory ID and the lot or serial number.)

`LocationState` should be cleared only on a hard reset—that is, on mode changes and when a user clicks the **Reset** button on the form toolbar—because the user does not enter a location on every cycle. (The location is stored in memory between cycles. Therefore, the location ID is optional if it was already entered.)

You don't want to clear the current document's number, even when a user clicks the **Reset** button, because the document defines the flow itself.

Process Overview

You will override the `ScanMode.ResetMode()` method. In the method, you will define the clearing condition for each state of the mode. To define the clearing condition, you will use the `when` pattern instead of the `if` pattern.

System Preparation

Before you begin creating a scan mode, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Define the required properties of the scan mode by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Define the input states of the scan mode by performing the *Barcode Scan States: To Create the Input State* and *Extension of Scan Components: To Extend Predefined Scan States for a Custom Form* prerequisite activities.

Step: Defining the Resetting Logic

In the `CountMode` class, define how the mode should be cleared as follows:

```
public sealed class CountMode : ScanMode
{
    ...

    protected override void ResetMode(bool fullReset)
    {
        Clear<RefNbrState>(when: fullReset && !Basis.IsWithinReset);
        Clear<LocationState>(when: fullReset);
        Clear<InventoryItemState>();
        Clear<LotSerialState>();
    }

    ...
```

```
}
```

Creating a Custom Barcode-Driven Form

In this chapter, you will learn how to create a custom barcode-driven Acumatica ERP form.

Custom Barcode-Driven Form: General Information

A user interacts with a barcode-driven form mostly via text commands that the user enters or scans into a special field of the form. The form guides the user through a certain input route by showing different input prompts.

Learning Objectives

In this chapter, you will learn how to create a custom barcode-driven form.

Applicable Scenarios

You create a custom barcode-driven form when you need to simplify the work with the form to the level of sequential text inputs.

Creation of a Custom Barcode-Driven Form

When you create a custom barcode-driven form, you perform the following steps:

1. Add a barcode scan class and implement its functionality, as demonstrated in *Barcode Scan Class: To Create a Barcode Scan Class*.
2. Create a scan mode and implement its required properties, as illustrated in *Barcode Scan Mode: To Define the Required Properties*.
3. To define the scan mode, do the following:
   a. Define the list of scan states for the scan mode, as demonstrated in *Barcode Scan States: To Create the Set of Scan States*.
   b. Define transition rules, as shown in *Barcode Scan Transitions: To Implement Transitions*.
   c. Define the list of commands, as illustrated in *Barcode Scan Commands: To Define the List of Commands*.
   d. Define the list of mode redirects, as shown in *Scan Mode Redirects: To Define the List of Redirects*.
   e. Define the resetting logic of the scan mode, as illustrated in *Reset of Barcode Scan Mode: To Define the Resetting Logic*.
4. Implement scan states as follows:
   a. Create an input state, as shown in *Barcode Scan States: To Create the Input State*.
   b. Create a confirmation state, as described in *Barcode Scan States: To Create the Confirmation State*.
   c. Alter the predefined scan states, as shown in *Extension of Scan Components: To Extend Predefined Scan States for a Custom Form*.
5. Implement scan commands by doing the following:
   a. Define custom commands, as described in *Barcode Scan Commands: To Add Quantity Support*.
   b. If you want to add logic that handles quantity input, add quantity support command, as specified in *Barcode Scan Commands: To Add Quantity Support*.
6. Add an ASPX page, as shown in *Custom Barcode-Driven Form: To Create an ASPX Page*.

ASPX Page of the Form

In the ASPX code of the page, you need to include the following required parts:

- The script that causes focus to be kept on the `Barcode` field's control
- The script that plays the sound that corresponds to the message type of the barcode-driven engine
- The `PXDataSource` control that binds to the host graph of the barcode scan class, but not to the barcode scan class itself

Custom Barcode-Driven Form: To Create an ASPX Page

This activity will walk you through the creation of an ASPX page for a custom barcode-driven form.

Story

Suppose that you have implemented the business logic of a custom barcode-driven form. Now you need to create an ASPX page for the form so that the form is available in the UI of Acumatica ERP.

Process Overview

You will create an ASPX page form from an ASPX template that contains the scripts required for a barcode-driven form to work and adjust this template for the custom form. You will also add the ASPX and ASPX.CS files to the customization project, add the form to the site map, and create the Site Map item for the form in the customization project.

System Preparation

Before you begin performing the steps of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Add a barcode scan class and implement its functionality by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create a scan mode and implement its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. To define the scan mode, do the following:
   a. Define the list of scan states for the scan mode by performing the *Barcode Scan States: To Create the Set of Scan States* prerequisite activity.
   b. Define transition rules by performing the *Barcode Scan Transitions: To Implement Transitions* prerequisite activity.
   c. Define the list of commands by performing the *Barcode Scan Commands: To Define the List of Commands* prerequisite activity.
   d. Define the list of mode redirects by performing the *Scan Mode Redirects: To Define the List of Redirects* prerequisite activity.
   e. Define the resetting logic of the scan mode by performing the *Reset of Barcode Scan Mode: To Define the Resetting Logic* prerequisite activity.
5. Implement scan states as follows:

a. Create an input state by performing the *Barcode Scan States: To Create the Input State* prerequisite activity.

b. Create a confirmation state by performing the *Barcode Scan States: To Create the Confirmation State* prerequisite activity.

c. Alter the predefined scan states by performing the *Extension of Scan Components: To Extend Predefined Scan States for a Custom Form* prerequisite activity.

6. Implement scan commands as follows:

   a. Define custom commands by performing the *Barcode Scan Commands: To Add Quantity Support* prerequisite activity.

   b. If you want to add logic that handles quantity input, add the quantity support command by performing the *Barcode Scan Commands: To Add Quantity Support* prerequisite activity.

Step 1: Creating the ASPX Page

To create the ASPX page for a custom barcode-driven form, do the following:

1. In the website project, which is the `PhoneRepairShop` project by default, add a new item named `RS302000.aspx` based on the *Web Form* template in the **PhoneRepairShop > Pages > RS** folder.

   The `RS302000.aspx` and `RS302000.aspx.cs` files are created.

2. In the `RS302000.aspx` file, replace the code with the following code.

   This is the minimum of markup needed to make a barcode-driven form work. You can use this piece of code as a template for creating any new barcode-driven ASPX page.

```
<%@ Page Language="C#" MasterPageFile="~/MasterPages/FormDetail.master"
 AutoEventWireup="true" ValidateRequest="false" CodeFile="RS302000.aspx.cs"
 Inherits="Page_IN305020" Title="Scan and Count" %>
<%@ MasterType VirtualPath="~/MasterPages/FormDetail.master" %>

<asp:content id="cont1" contentplaceholderid="phDS" runat="Server">
    <script type="text/javascript">
        function Barcode_Initialize(ctrl) { // This script makes focus to be kept on
 the Barcode field's control.
            ctrl.element.addEventListener('keydown', function (e) {
                if (e.keyCode === 13) { // Enter key
                    e.preventDefault();
                    e.stopPropagation();
                }
            });
        };
    </script>

    <script type="text/javascript">
        function ActionCallback(callbackContext) { // This script plays the sound that
 corresponds to the message type of the barcode-driven engine.
            var baseUrl = (location.href.indexOf("HideScript") > 0) ? "../../
Sounds/" : "../../../Sounds/";
            var edInfoMessageSoundFile = px_alls["edInfoMessageSoundFile"];

            if ((callbackContext.info.name.toLowerCase().startsWith("scan")
 || callbackContext.info.name == "ElapsedTime") &&
 callbackContext.control.longRunInProcess == null && edInfoMessageSoundFile != null) {
                var soundFile = edInfoMessageSoundFile.getValue();
```

```
                if (soundFile != null && soundFile != "") {
                    var audio = new Audio(baseUrl + soundFile + '.wav');
                    audio.play();
                }
            }
        };
        window.addEventListener('load', function ()
 { px_callback.addHandler(ActionCallback); });
    </script>

    <%-- Note that the datasource binds to the host graph of the barcode scan class --
%>
    <px:PXDataSource ID="ds" runat="server" TypeName="PhoneRepairShopt.INScanCount
+Host" PrimaryView="HeaderView">
        <CallbackCommands>
        </CallbackCommands>
    </px:PXDataSource>
</asp:content>

<asp:content id="cont2" contentplaceholderid="phF" runat="Server">
    <px:PXFormView ID="formHeader" runat="server" DataSourceID="ds" Height="120px"
 Width="100%" DataMember="HeaderView" DefaultControlID="edBarcode">
        <Template>
            <px:PXLayoutRule runat="server" StartColumn="True" LabelsWidth="S"
 ControlSize="L" />
            <px:PXTextEdit ID="edBarcode" runat="server" DataField="Barcode">
                <AutoCallBack Command="Scan" Target="ds">
                    <Behavior CommitChanges="True" />
                </AutoCallBack>
                <ClientEvents Initialize="Barcode_Initialize"/>
            </px:PXTextEdit>

            <px:PXLayoutRule runat="server" StartColumn="True" LabelsWidth="S"
 ControlSize="L" ColumnWidth="M" />
            <px:PXTextEdit ID="edMessage" runat="server" DataField="Message"
                Height="55px" Width="800px" Style="font-size: 10pt; font-weight:
 bold;"
                SuppressLabel="true" TextMode="MultiLine" SkinID="Label"
 DisableSpellcheck="True" Enabled="False" />
        </Template>
    </px:PXFormView>
    <px:PXFormView ID="formInfo" runat="server" DataSourceID="ds" DataMember="Info">
        <Template>
            <px:PXTextEdit ID="edInfoMode" runat="server" DataField="Mode"/>
            <px:PXTextEdit ID="edInfoMessage" runat="server" DataField="Message"/>
            <px:PXTextEdit ID="edInfoMessageSoundFile" runat="server"
 DataField="MessageSoundFile"/>
            <px:PXTextEdit ID="edInfoPrompt" runat="server" DataField="Prompt"/>
        </Template>
    </px:PXFormView>
</asp:content>

<asp:content id="cont3" contentplaceholderid="phG" runat="Server">
</asp:content>
```

3. Add the `RefNbr` and `SiteID` fields after the `Barcode` field.

```
<px:PXSelector ID="edRefNbr" runat="server" DataField="RefNbr" AllowEdit="true" />
<px:PXSelector ID="edSiteID" runat="server" DataField="SiteID" AllowEdit="true" />
```

4. Add a tab item with a grid that shows physical inventory details to the third content placeholder.

```
<px:PXTab ID="tab" runat="server" Height="540px" Style="z-index: 100;" Width="100%">
    <Items>
        <px:PXTabItem Text="Count">
            <Template>
                <px:PXGrid ID="gridPIDetail" runat="server" DataSourceID="ds"
 SyncPosition="true" Width="100%" SkinID="DetailsInTab">
                    <Levels>
                        <px:PXGridLevel DataMember="PIDetail">
                            <Columns>
                                <px:PXGridColumn DataField="LineNbr" />
                                <px:PXGridColumn DataField="Status" />
                                <px:PXGridColumn DataField="LocationID" />
                                <px:PXGridColumn DataField="InventoryID" />
                                <px:PXGridColumn
 DataField="InventoryID_InventoryItem_descr"/>
                                <px:PXGridColumn DataField="LotSerialNbr" />
                                <px:PXGridColumn DataField="BookQty"
 TextAlign="Right" />
                                <px:PXGridColumn DataField="PhysicalQty"
 TextAlign="Right" />
                                <px:PXGridColumn DataField="VarQty"
 TextAlign="Right" />
                            </Columns>
                            <RowTemplate>
                                <px:PXLayoutRule runat="server" StartColumn="True"
 LabelsWidth="M" ControlSize="XM" />
                                <px:PXSegmentMask ID="edLocationID" runat="server"
 DataField="LocationID" AutoRefresh="True" />
                                <px:PXNumberEdit ID="edBookQty" runat="server"
 DataField="BookQty" />
                                <px:PXNumberEdit ID="edPhysicalQty" runat="server"
 DataField="PhysicalQty" />
                                <px:PXNumberEdit ID="edVarQty" runat="server"
 DataField="VarQty" />
                            </RowTemplate>
                        </px:PXGridLevel>
                    </Levels>
                    <AutoSize Container="Window" Enabled="True" MinHeight="400" />
                </px:PXGrid>
            </Template>
        </px:PXTabItem>
    </Items>
    <AutoSize Enabled="True" Container="Window" />
</px:PXTab>
```

5. Optional: Add another tab item that displays the logs of the barcode-driven engine.

```
<px:PXTabItem Text="Scan Log">
    <Template>
        <px:PXGrid ID="gridScanLog" runat="server" DataSourceID="ds" Style="height:
 250px;" Width="100%" SkinID="Inquire" Height="372px" >
            <Levels>
```

```
                <px:PXGridLevel DataMember="Logs">
                    <Columns>
                        <px:PXGridColumn DataField="ScanTime" />
                        <px:PXGridColumn DataField="Mode" />
                        <px:PXGridColumn DataField="Prompt" />
                        <px:PXGridColumn DataField="Scan" />
                        <px:PXGridColumn DataField="Message" />
                    </Columns>
                </px:PXGridLevel>
            </Levels>
            <AutoSize Container="Window" Enabled="True" MinHeight="400" />
        </px:PXGrid>
    </Template>
</px:PXTabItem>
```

6. Save your changes.
7. In the `RS302000.aspx.cs` file, replace the code with the following code.

```
using System;
using System.Drawing;
using System.Web.UI.WebControls;
using PX.Data;
using PX.BarcodeProcessing;

public partial class Page_IN305020 : PX.Web.UI.PXPage
{
 private static class PickCss
 {
  public const string Complete = "CssComplete";
  public const string Partial = "CssPartial";
  public const string Overpick = "CssOverpick";
 }

 protected void Page_Init(object sender, EventArgs e) { }

 protected void Page_Load(object sender, EventArgs e)
 {
  RegisterStyle(PickCss.Complete, null, Color.Green, true);
  RegisterStyle(PickCss.Partial, null, Color.Black, true);
  RegisterStyle(PickCss.Overpick, null, Color.OrangeRed, true);
 }

 private void RegisterStyle(string name, Color? backColor, Color? foreColor, bool
 bold)
 {
  Style style = new Style();
  if (backColor.HasValue) style.BackColor = backColor.Value;
  if (foreColor.HasValue) style.ForeColor = foreColor.Value;
  if (bold) style.Font.Bold = true;
  this.Page.Header.StyleSheet.CreateStyleRule(style, this, "." + name);
 }

 protected void LogGrid_RowDataBound(object sender, PX.Web.UI.PXGridRowEventArgs e)
 {
  var log = PXResult.UnwrapMain(e.Row.DataItem);
  if (log is ScanLog bdsmLog)
  {
```

```
    if (bdsmLog.MessageType == ScanMessageTypes.Error)
     e.Row.Style.CssClass = PickCss.Overpick;
    else if (bdsmLog.MessageType == ScanMessageTypes.Warning)
     e.Row.Style.CssClass = PickCss.Partial;
   }
  }
 }
```

Step 2: Adding ASPX and ASPX.CS Files to the Customization Project

To add the new files to the customization project, do the following:

1. Open the *PhoneRepairShop* customization project in the Customization Project Editor.
2. In the navigation pane, click **Files**. The *Files* page opens.
3. On the page toolbar, click **Add New Record**.
4. In the **Add Files** dialog box, select the unlabeled check boxes for the `Pages\RS\RS302000.aspx` and `Pages\RS\RS302000.aspx.cs` items, and click **Save**.

Step 3: Adding the Form to the Site Map

To be able to see the new form in the UI of Acumatica ERP, you need to add the new form to the site map. Do the following:

1. On the toolbar of the *Site Map* (SM200520) form, click **Add Row**.
2. In the new row, specify the following settings:
   - **Screen ID**: `RS302000`
   - **Title**: `Scan and Count`
   - **URL**: `~/Pages/RS/RS302000.aspx`
   - **Workspaces**: *Phone Repair Shop*
   - **Category**: *Automated Operations*
3. Save your changes and close the form.

Step 4: Adding the Site Map Item to the Customization Project

To add to the customization project the site map item that corresponds to the form, do the following:

1. Open the *PhoneRepairShop* customization project in the Customization Project Editor.
2. On the navigation pane, click **Site Map** to open the *Site Map* page.
3. On the page toolbar, click **Add New Record**.
4. In the **Add Site Map** dialog box, which is opened, find the row with the *RS302000* screen ID, and select the unlabeled check box in that row.
5. Click **Save**.

Extending Scan Components

In this chapter, you will learn how to customize scan components of existing forms. You will also learn how to alter predefined components before adding them to a custom form.

Extension of Scan Components: General Information

The `WarehouseManagementSystem<TSelf, TGraph>` class has multiple predefined components that can be used by its descendants. However, sometimes you need to alter the logic of predefined components before you add them to the components used in the mode you are developing. You can also customize the scan components that are used on existing barcode-driven forms of Acumatica ERP.

Learning Objectives

In this chapter, you will learn how to do the following:

- Customize a scan component that is used on an existing barcode-driven Acumatica ERP form
- Add a new component that alters a predefined scan component, and use it on a custom barcode-driven form

Applicable Scenarios

You extend a scan component in the following cases:

- You need to modify the logic of a scan component of an existing barcode-driven Acumatica ERP form for your business scenario. For example, you may want to change the order of value input, implement additional entity validation, or add an alternate way to search for entities.
- You have created a scan mode and have inherited a scan component of this mode from the base barcode scan class. You need to modify this component to use it in your scan mode. For example, you may want to add additional validation that is specific to this scan mode.

Extension of Scan Components

Scan components use a method-intercepting approach, which predefines the extendable parts of the components. You can learn about method interceptors in *Extension of Scan Components: Method Interceptors*.

A scan component is called simple if it implements all its logic by itself. A component is called complex if it does not implement all its logic by itself and delegates logic implementation to at least one `ScanExtension` class. For details about scan extensions, see *Extension of Scan Components: Scan Extensions*.

Extension of Predefined Components for a Custom Form

To create a component that alters a predefined scan component and use it on a custom form, you do one of the following:

- Introduce a descendant of a predefined scan component.

  You can introduce a descendant of a component that contains all changes and is nested in this scan mode. You can use the same name for the descendant.

  By introducing a descendant, you merge the logic of the base class and the logic of its descendant. Therefore, there is no way to override the logic of the base class and keep the logic added by its descendant, even through the use of the `ByBaseSubstitute` interception strategy. (For details about method interceptors, see *Extension of Scan Components: Method Interceptors*.) Also, the descendant-based approach

uses the classic overriding technique and thus cannot use simplified overriding strategies that are available in the following approaches.

- Inject the logic when the component is created.

  You can inject the needed changes right inside the component instance when the component is created.

  The method interception approach provides multiple overriding strategies that could facilitate logic altering. When you override the `Create` method that corresponds to the component (such as the `CreateStates()` method for the scan state component), all logic patches made by this approach can be suppressed. Moreover, with the `ByBaseSubstitute` interception strategy, you can change the base logic of a component without affecting already applied logic patches.

- Inject the logic when the component is decorated.

  You can inject desired changes right inside the component instance when the component is decorated.

  When a component's logic gets intercepted during component decoration, it does not matter which code creates this component. This means that patches made this way would stay effective even if the component creation method of a scan mode was completely replaced with another one providing the same components. However, patches made with this approach can still be suppressed or extended if they were extracted to a `virtual` method of the barcode scan class itself or of one of its extensions.

  > We recommend that you always override the methods of the `Decorate` family in full form (that is, by accepting the base method as the last parameter), call the base implementation, and use its result. Otherwise, you will lose all potential changes that could be made by other customization projects.

For examples of altering predefined scan state components, see *Extension of Scan Components: To Extend Predefined Scan States for a Custom Form*.

Extension of Scan Components of an Existing Form

To customize the logic of a component, you override the needed `Create` or `Decorate` method by using the `PXOverride` attribute. You also intercept one of the component members that can be intercepted. For details on the customization of particular components, see *Extension of Scan Components: Customization of Components*.

For examples of customizing components, see the following topics:

- *Extension of Scan Components: To Add an Alternate Way to Search for Entities*
- *Extension of Scan Components: To Implement Additional Entity Validation*
- *Extension of Scan Components: To Add a New Scan Command for an Existing Form*
- *Extension of Scan Components: To Change the Order of Value Input*

Extension of Scan Components: Method Interceptors

A method interception approach provides multiple overriding strategies that could facilitate the customization process.

Method Interceptors

All components of `BarcodeDrivenStateMachine` should be either `abstract` or `sealed`. An `abstract` class should be used as a template for a `sealed` class. Only `sealed` classes can be used as scan components. These components should not contain any customizable logic. All customizable logic should be located in the `Basis` class or in its extensions (which are `ScanExtension` descendants). You perform the configuration of the components by using method interceptors.

A method interceptor is a special class that can simulate inheritance and overriding in the scope of a particular method. The following diagram shows the classes related to method interceptors.

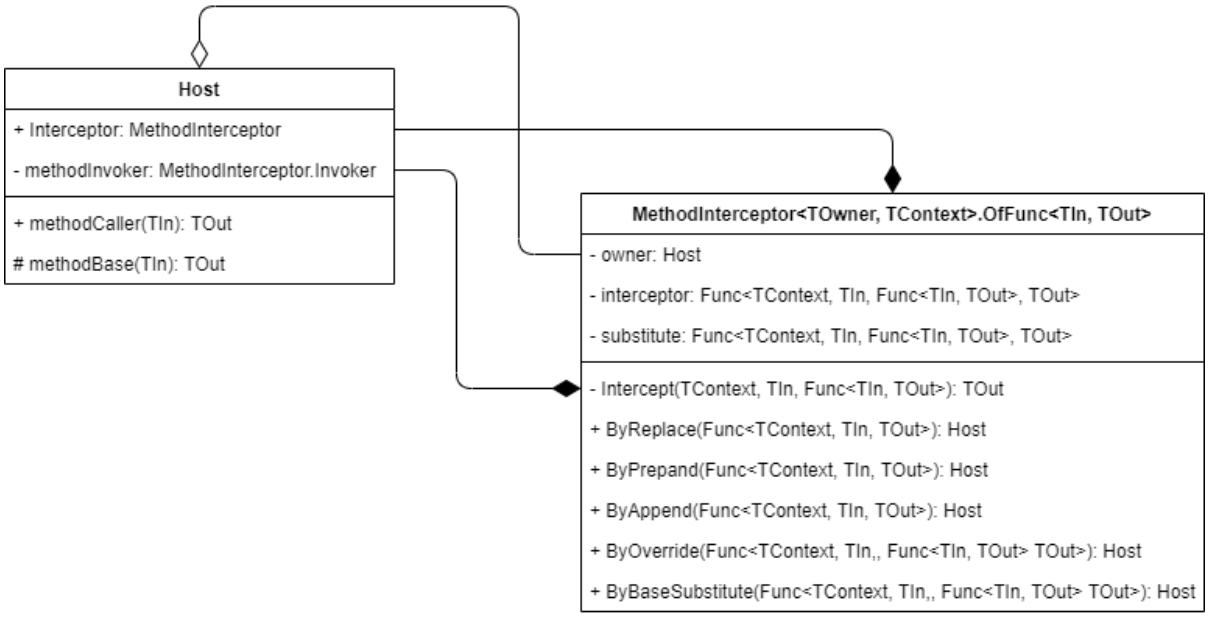


Figure: Method interceptor

A host class of the interceptor class creates a method interceptor for one of its methods by passing the base method to the interceptor. The host provides the public access to the interceptor so that the interception can be performed by the external classes (via the `By` method group). The host calls the target method only through the interceptor's method invoker, which is provided by the interceptor and is not accessible for external code. The host may have an additional `public` method that makes it possible for external code to use the target method properly.

Interception Approaches

The barcode-driven engine provides the following interception approaches:

- Override: This approach is similar to an original overriding that is used in object-oriented programming except that the base method is passed as the last parameter of the overriding method. A base method is a multicast delegate that accumulates all injected delegates that have been applied previously.
- Replace: This approach simplifies the override strategy. The base method and all previously injected delegates are not invoked because the replacing delegate replaces them completely.
- Prepend: This approach simplifies of the override strategy. The base method and all previously injected delegates are automatically invoked after the prepended delegate.
- Append: This approach simplifies of the override strategy. The base method and all previously injected delegates are automatically invoked before the appended delegate.
- Base substitution: This approach overrides only the original method of a component and does not affect any injected delegates.

All approaches can accept two versions of related delegates—that is, with or without `basis` as the first parameter. Although a delegate can capture the `Basis` property of the `ScanMode` instance or the `this` reference of the `BarcodeDrivenStateMachine` instance, we recommend that you use the `basis` parameter of the delegate instead. This helps to minimize extra memory allocations that are produced by C# closures.

The following example shows additional validation for a certain entity state.

```
protected override ScanState<MyBarcodeExt> DecorateScanState(ScanState<MyBarcodeExt>
 original)
```

```
{
    var state = base.DecorateScanState(original); // all other interceptions should also
 take place

    if (state is MySomeState myState)
    {
        myState
            .Intercept.Validate.ByAppend((basis, entity) =>
            {
                // use the basis parameter to avoid variable capture
                if (basis.SomeSetup.Current.SomeRestriction == true &&
                    entity.SomeFlag == false)
                    return Validation.Fail("Some reason");
                return Validation.Ok;
            });
    }

    return state;
}
```

> Within a patch method, only the `basis` parameter should be used—that is, we recommend that you do not use the `Basis` property in this method. Otherwise, the compiler does not create a `static` lambda method, which leads to unnecessary memory allocation.

The replace strategy of a method interceptor is not compatible with any other interceptions made by other extensions. Therefore, we recommend that you use it only when you are creating the topmost extension. In other cases, use the override approach—that is, get the result of the base method and use it to produce the changes you want to achieve. The replace strategy is easier to use, while the override approach is more flexible and makes further customization possible.

Extension of Scan Components: Scan Extensions

Because all scan components have to be `sealed` classes, they cannot have `virtual` methods and thus cannot be customized through class inheritance. Components use method interceptors for customizations instead. A component can provide its own `ScanExtension` class where all `virtual` methods are placed.

Scan Extension

The `ScanExtension` classes are `PXGraphExtension` descendants, which can simplify the creation of customizable logic that should be used in the scope of barcode processing. For example, if a `ScanCommand<TScanBasis>` descendant must have complex logic that should be split into multiple methods, you should create a `ScanExtension` descendant that holds all these methods. This extension can be placed inside the `ScanCommand<TScanBasis>` descendant so that its customizable logic is not mixed with the logic of other components.

The `ScanExtension` descendant can be customized as any other `PXGraphExtension` can via `PXOverrideAttribute`. Any component can access the extension by using the `Basis.Get<TScanExtension>()` method, which is a shortcut for the `Basis.Graph.FindImplementation<TScanExtension>()` call.

When you create an extension for a scan component, you can use a `PXGraphExtension<>` class descendant instead of `ScanExtension`. However, in this case, the definition of the extension lacks the `Basis` property that is used by all scan components. You need to define this property manually, as shown in the following code.

```
public class BoycottAntarcticaItems : PXGraphExtension<ReceivePutAway,
    ReceivePutAway.Host>
{
    public ReceivePutAway Basis => Base1;
}
```

The classes related to scan extensions are shown in the following diagram.

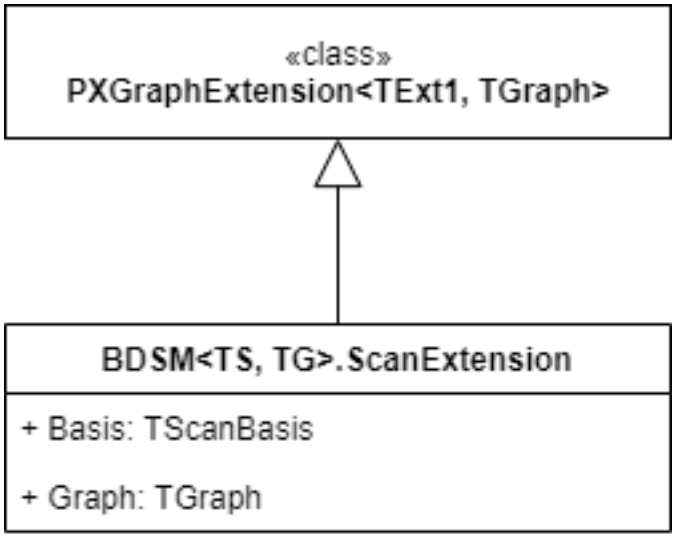


Figure: Scan extension relations

Example of the ScanExtension Approach

The following code shows the initial definition of a scan component.

```
public class MyBarcodeExt : BarcodeDrivenStateMachine<MyBarcodeExt, Host>
{
    public class Host : TargetGraph { }

    // All overridable logic is moved to the Logic class
    public sealed class MyCommand : ScanCommand
    {
        protected override bool Process()
        {
            Basis.Get<Logic>().Foo();
            return true;
        }

        public class Logic : ScanExtension
        {
            public virtual void Foo()
            {
                int bar = Bar("madskillz");
                Basis.DoBar(bar);
```

```
            }

            public virtual int Bar(string input)
            {
                return 7;
            }
        }
    }
}
```

The following code shows the customization.

```
// Overrides MyBarcodeExt.MyCommand.Logic
public class SomeCustomization : PXGraphExtension<
    MyBarcodeExt.MyCommand.Logic, // the logic you want to customize
    MyBarcodeExt, // optional, is useful if Basis is needed to be used in scope of the
 customization
    MyBarcodeExt.Host // an actual graph
>
{
    public MyBarcodeExt Basis => Base1;
    // Overrides MyBarcodeExt.MyCommand.Logic.Bar(string)
    [PXOverride]
    public virtual int Bar(string input, Func<string, int> base_Bar)
    {
        return base_Bar(input) + 42;
    }
}
```

ScanExtension<TScanExtension> Class

The `ScanExtension<TScanExtension>` class is a `ScanExtension` component that is parameterized with another `ScanExtension` component type. This class is useful if you want to customize a `ScanExtension` component.

For example, you can define an extension for the `INScanCount.CountMode.ConfirmState.Logic` class in the following ways:

- By using the `PXGraphExtension<>` class

```
public class AlterCountModeConfirmStateLogic :
    PXGraphExtension<INScanCount.CountMode.ConfirmState.Logic,
    INScanCount, INScanCount.Host>
{
    // some logic
}
```

- By using the `ScanExtension<>` class

```
public class AlterCountModeConfirmStateLogic :
    INScanCount.ScanExtension<INScanCount.CountMode.ConfirmState.Logic>>
{
    // some logic
}
```

Although you can use the way you prefer, we recommend that you use the second one.

Extension of Scan Components: To Extend Predefined Scan States for a Custom Form

This activity will walk you through the process of altering predefined scan states.

Story

Suppose that you are creating a scan mode for a custom barcode-driven form. You have inherited the following input states from the base barcode scan class: `LocationState`, `InventoryItemState`, and `LotSerialState`. You need to modify these states to use them in your scan mode as follows:

- For the `LocationState` scan state, you want to add additional validation that is specific to this current scan mode.
- For the `InventoryItemState` scan state, you want to add an additional absence handler that is specific to this scan mode.
- You want the `LotSerialState` to be active only when it is already active and when the lot or serial number is entered on receipt.

Process Overview

You will implement the following ways of altering the logic of the predefined components:

- You will introduce a descendant for the `LocationState` class.
- You will inject the logic when `InventoryItemState` is created.
- You will inject the logic when `LotSerialState` is decorated.

System Preparation

Before you begin performing the steps of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Create a barcode scan class by performing the *Barcode Scan Class: To Create a Barcode Scan Class* prerequisite activity.
3. Create the scan mode and define its required properties by performing the *Barcode Scan Mode: To Define the Required Properties* prerequisite activity.
4. Create the set of scan states by performing the *Barcode Scan States: To Create the Set of Scan States* prerequisite activity.

Step 1: Introducing a Descendant

To alter a scan state by introducing a descendant, do the following:

1. In the `CountMode` class, define the `LocationState` class, as follows.

```
public class INScanCount : WMSBase
{
    public sealed class CountMode : ScanMode
    {
        ...
        public new sealed class LocationState : WMSBase.LocationState
```

```
        {

        }
        ...
    }
}
```

You use the same name for the descendant that is used for the predefined class. The code of this activity includes two versions of the `LocationState` class: the original `WMSBase.LocationState` class and the `CountMode`-related `INScanCount.CountMode.LocationState` class. Because the original class and the descendant have the same name, you will not need to change the `CreateStates()` method.

2. In the `LocationState` class, implement the `IsStateSkippable()` method so that `LocationState` is skipped if the location is already selected, as shown in the following code.

```
public new sealed class LocationState : WMSBase.LocationState
{
    protected override bool IsStateSkippable() => Basis.LocationID != null;
}
```

In `CountMode`, you do not need to ask a user for a location again if the location is already selected. Therefore, you implement the `IsStateSkippable()` method so that `LocationState` is skipped if the location is already selected.

3. In the `INScanCount.cs`, add the following `using` directive.

```
using PX.Data.BQL.Fluent;
```

4. In the `LocationState` class, add the `Validate()` method, as shown in the following code.

```
public new sealed class LocationState : WMSBase.LocationState
{
    protected override bool IsStateSkippable() => Basis.LocationID != null;

    protected override Validation Validate(INLocation location)
    {
        if (Basis.HasFault(location, base.Validate, out Validation fault))
            return fault;

        INPIStatusLoc statusLocation =
            SelectFrom<INPIStatusLoc>.
            Where<
                INPIStatusLoc.pIID.IsEqual<WMSScanHeader.refNbr.FromCurrent>.
                And<
                    INPIStatusLoc.locationID.IsEqual<@P.AsInt>.
                    Or<INPIStatusLoc.locationID.IsNull>>>.
            View.ReadOnly
            .Select(Basis, location.LocationID);

        if (statusLocation == null)
            return Validation.Fail(Msg.NotPresent, location.LocationCD);

        return Validation.Ok;
    }

    [PXLocalizable]
    public new abstract class Msg : WMSBase.LocationState.Msg
    {
```

```
        public const string NotPresent = "The {0} location is not in the list and
 cannot be added.";
    }
}
```

In `CountMode`, you need only locations that are present in the physical inventory that the user is currently processing. Therefore, you add additional validation logic in the `Validate()` method. You execute the base validation first. The custom validation is performed only if the base validation is passed.

Step 2: Injecting the Logic When the Component Is Created

To alter a scan state by injecting the logic when the scan state is created, in the `CountMode` class, modify the `CreateStates()` method as follows.

```
public class INScanCount : WMSBase
{
    public sealed class CountMode : ScanMode
    {
        ...
        protected override IEnumerable<ScanState<INScanCount>> CreateStates()
        {
            yield return new RefNbrState();
            yield return new LocationState();
            yield return new InventoryItemState()
                .Intercept.HandleAbsence.ByAppend((basis, barcode) =>
                {
                    if (basis.TryProcessBy<LocationState>(barcode,
                        StateSubstitutionRule.KeepPositiveReports |
                        StateSubstitutionRule.KeepApplication))
                        return AbsenceHandling.Done;
                    return AbsenceHandling.Skipped;
                });
            yield return new LotSerialState();
            yield return new ConfirmState();
        }
        ...
    }
}
```

In the `CreateStates()` method, you intercept the `HandleAbsence` method. You use the `ByAppend` interception strategy because you want to register an additional absence handler without affecting the existing ones. In `CountMode`, you need the ability to switch the location even if the current input state is `InventoryItemState`. You also configure the `LocationState` state to try handle the input.

Step 3: Injecting the Logic When the Component Is Decorated

To alter a scan state by injecting the logic when the scan state is decorated, in the `CountMode` class, add the `DecorateScanState()` method as follows.

```
public class INScanCount : WMSBase
{
    ...
    protected override ScanState<INScanCount> DecorateScanState(
        ScanState<INScanCount> original)
    {
        var state = base.DecorateScanState(original);
```

```
        if (state is WMSBase.LotSerialState lotSerialState)
            InjectLotSerialDeactivationOnNonEnterableLotSerials(lotSerialState);

        return state;
    }

    protected virtual void InjectLotSerialDeactivationOnNonEnterableLotSerials(
        WMSBase.LotSerialState lotSerialState)
    {
        lotSerialState
            .Intercept.IsStateActive.ByConjoin(
                basis => basis.LotSerialTrack.IsEnterable);
    }
    ...
}
```

In this code, you will intercept the `IsStateActive` method and use the `ByConjoin` interception strategy.

Extension of Scan Components: Customization of Components

In this topic, you can find information about how to customize particular scan components.

Set of Scan Modes

If you need to change the set of modes of the barcode-driven engine, such as when you are adding a new mode, you override the `CreateScanModes` method by using the `PXOverride` attribute and add your new mode, as shown in the following code.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
    [PXOverride]
    public virtual IEnumerable<ScanMode<SomeWMS>>
 CreateScanModes(Func<IEnumerable<ScanMode<SomeWMS>>> base_CreateScanModes)
    {
        foreach (var mode in base_CreateScanModes())
            yield return mode;

        yield return new MyMode();
    }

    public sealed class MyMode : SomeWMS.ScanMode
    {
        // implement a new mode here
    }
}
```

Scan Mode

If you need to change the configuration of the scan mode, you override the `DecorateScanMode` method by using the `PXOverride` attribute and intercept the logic of the mode in this method, as shown in the following code.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
```

```
    [PXOverride]
    public virtual ScanMode<SomeWMS> DecorateScanMode(
        ScanMode<SomeWMS> original,
        Func<ScanMode<SomeWMS>, ScanMode<SomeWMS>> base_DecorateScanMode)
    {
        var mode = base_DecorateScanMode(original);

        if (mode is SomeMode someMode)
        {
            mode
                .Intercept.CreateStates.ByOverride((basis, base_CreateStates) =>
                {
                    // Change the set of states
                })
                .Intercept.CreateTransitions.ByReplace(basis =>
                {
                    // Change state-dispatching rules
                })
                .Intercept.CreateCommands.ByAppend(basis =>
                {
                    // Add additional commands
                })
                .Intercept.CreateQuestions.ByBaseSubstitute((basis, base_CreateQuestions)
 =>
                {
                    // Change the original set of questions
                })
                .Intercept.GetDefaultState.ByOverride((basis, base_GetDefaultState) =>
                {
                    // Change the default state
                })
                .Intercept.ResetMode.ByPrepend((basis, fullReset) =>
                {
                    // Change the state-clearing logic
                });
        }

        return mode;
    }
}
```

Scan Command

If you need to change the simple configuration logic of the scan command, you override the `DecorateScanCommand` method by using the `PXOverride` attribute and intercept the logic of the component in this method, as shown in the following code.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
    [PXOverride]
    public virtual ScanCommand<SomeWMS> DecorateScanCommand(
        ScanCommand<SomeWMS> original,
        Func<ScanCommand<SomeWMS>, ScanCommand<SomeWMS>> base_DecorateScanCommand)
    {
        var command = base_DecorateScanCommand(original);

        if (command is SomeCommand someCommand)
```

```
        {
            someCommand
                .Intercept.IsEnabled.ByConjoin(basis =>
                {
                    // Add a condition when the command is enabled
                })
                .Intercept.Process.ByOverride((basis, base_Process) =>
                {
                    // Change the logic of the command
                });
        }

        return command;
    }
}
```

Mode Redirect

If you need to change the simple configuration logic of the mode redirect, you override the `DecorateScanRedirect` method by using the `PXOverride` attribute and intercept the logic of the component in this method, as shown in the following code.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
    [PXOverride]
    public virtual ScanRedirect<SomeWMS> DecorateScanRedirect(
        ScanRedirect<SomeWMS> original,
        Func<ScanRedirect<SomeWMS>, ScanRedirect<SomeWMS>> base_DecorateScanRedirect)
    {
        var redirect = base_DecorateScanRedirect(original);

        if (redirect is SomeRedirect someRedirect)
        {
            someRedirect
                .Intercept.PrepareRedirect.ByOverride((basis, base_PrepareRedirect) =>
                {
                    // Redirect-preparing logic
                })
                .Intercept.CompleteRedirect.ByOverride((basis, base_CompleteRedirect) =>
                {
                    // Redirect-completing logic
                });
        }

        return redirect;
    }
}
```

Scan Question

If you need to change the simple configuration logic of the scan question, you override the `DecorateScanQuestion` method by using the `PXOverride` attribute and intercept the logic of the componentin this method, as shown in the following code.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
    [PXOverride]
```

```
    public virtual ScanQuestion<SomeWMS> DecorateScanQuestion(
        ScanQuestion<SomeWMS> original,
        Func<ScanQuestion<SomeWMS>, ScanQuestion<SomeWMS>> base_DecorateScanQuestion)
    {
        var question = base_DecorateScanQuestion(original);

        if (question is SomeQuestion someQuestion)
        {
            someQuestion
                .Intercept.GetPrompt.ByReplace(basis => // new prompt)
                .Intercept.Confirm.ByOverride((basis, base_Confirm) =>
                {
                    // question-confirming logic
                })
                .Intercept.Reject.ByOverride((basis, base_Reject) =>
                {
                    // question-rejecting logic
                });
        }

        return question;
    }
}
```

Scan State

If you need to change the simple configuration logic of the scan state, you override the `DecorateScanState` method by using the `PXOverride` attribute and intercept the logic of the component in this method, as shown in the following code.

> For a multi-mode barcode-driven form, such as *Pick, Pack, and Ship* (SO302020) or *Receive and Put Away* (PO302020), each mode of the form can have its own `InventoryItemState` (or any other state) component. Therefore, we recommend that you always check for the mode of the desired component, unless you want to patch a component despite the mode it is used in.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
    [PXOverride]
    public virtual ScanState<SomeWMS> DecorateScanState(
        ScanState<SomeWMS> original,
        Func<ScanState<SomeWMS>, ScanState<SomeWMS>> base_DecorateScanState)
    {
        var state = base_DecorateScanState(original);

        if (state is SomeEntityState someEntityState)
        {
            someEntityState
                .Intercept.StatePrompt.ByReplace(
                    basis => // New prompt
                )
                .Intercept.IsStateActive.ByConjoin(
                    basis => // New conjoint (&&) condition
                ) // Prefer conjoin for this member
                .Intercept.IsStateSkippable.ByDisjoin(
                    basis => // New disjoint (||) condition
                ) // Prefer disjoin for this member
```

```
                .Intercept.GetByBarcode.ByOverride(
                    (basis, barcode, base_GetByBarcode) =>
                {
                    // Change entity-fetching logic
                })
                .Intercept.HandleAbsence.ByAppend((basis, barcode) =>
                {
                    // Handle the entity absence
                })
                .Intercept.ReportMissing.ByReplace((basis, barcode) =>
                {
                    // Report that the entity is missing
                })
                .Intercept.Validate.ByPrepend((basis, entity) =>
                {
                    // Validate the found entity
                })
                .Intercept.Apply.ByAppend((basis, entity) =>
                {
                    // Entity-application logic
                })
                .Intercept.ClearState.ByAppend((basis, entity) =>
                {
                    // Clear the changes made by the Apply method
                })
                .Intercept.SetNextState.ByOverride((basis, base_SetNextState) =>
                {
                    // Set the next state                })
                .Intercept.OnTakingOver.ByAppend(basis =>
                {
                    // Do something when the state takes over

                })
                .Intercept.OnDismissing.ByPrepend(basis =>
                {
                    // Do something when the state gets dismissed
                });
        }
        return state;
    }
}
```

Complex Logic

If you need to change the complex logic used by the barcode-driven engine or by any of its components, you create a nested extension that overrides the member of the particular `ScanExtension` component.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
    public class AlterSomeModeSomeCommandLogic :
 SomeWMS.ScanExtension<SomeWMS.SomeMode.SomeCommand.Logic>
    {
        /// <summary>
        /// Overrides <see cref="SomeWMS.SomeMode.SomeCommand.Logic.Bar(int)"/>
        /// </summary>
        [PXOverride]
        public virtual void Bar(int input, Action<int> base_Bar)
```

```
        {
            // change the logic of Bar method
        }
    }
}

// where the original code is:
public class SomeWMS : WarehouseManagementSystem<SomeWMS, SomeWMS.Host>
{
    public class Host : SomeGraph { }

    // WMS-related methods
    // ...

    public sealed class SomeMode : ScanMode
    {
        // mode-related methods
        // ...

        public sealed class SomeCommand : ScanCommand
        {
            // command-related methods
            // ...

            protected override bool Process()
            {
                Basis.Get<Logic>().DoSomething();
                return true;
            }

            public class Logic : ScanExtension // contains some complex logic that may be
 customized in parts
            {
                public virtual void DoSomething()
                {
                    // ...
                    Bar(42);
                    // ...
                }

                public virtual void Bar(int input)
                {
                    // some logic here that may call some other methods
                }
            }
        }
    }
}
```

Extension of Scan Components: To Change the Order of Value Input

This activity will walk you through changing the order of value input.

Story

Suppose that you need to alter the default order of value input of the Pick mode of the *Pick, Pack, and Ship* (SO302020) form from the one listed in the first column of the following table to the one listed in the second column.

Table: The Order of Input

Initial Order	Target Order
• The reference number of the shipment • The location from which the item is picked • The item being picked • The lot or serial number of the item • The expiration date of the item	• The reference number of the shipment • The item being picked • The lot or serial number of the item • The expiration date of the item • The location from which the item is picked

Process Overview

You will change the transition map for the Pick scan mode. To do this, you will create a scan extension for the `PX.Objects.SO.WMS.PickPackShip` class, override its `DecorateScanMode` method by using the `PXOverride` attribute, and intercept the `CreateTransitions` method in this method.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Enable the following features on the *Enable/Disable Features* (CS100000) form, where they are in the *Inventory and Order Management* group of features:
   - *Multiple Warehouse Locations*
   - *Warehouse Management*
   - *Fulfillment*

Step: Changing the Order of Value Input

To change the order of value input, do the following:

1. In the `PhoneRepairShop_Code` Visual Studio project, add a reference to the `PX.BarcodeProcessing.dll` file.
2. Create the `LocationGoesLastInPickMode` class, and make it `public`.
3. Add the `using` directives shown in the following code to the file.

```
using System;
using PX.Data;
using PX.BarcodeProcessing;
using PX.Objects.SO.WMS;
using PX.Objects.IN.WMS;
```

4. Make the class an extension of the `PX.Objects.SO.WMS.PickPackShip` class.

```
using WMSBase = WarehouseManagementSystem<PickPackShip, PickPackShip.Host>;
```

```
public class LocationGoesLastInPickMode : PickPackShip.ScanExtension
{
}
```

5. Override the `DecorateScanMode` method.

```
public class LocationGoesLastInPickMode : PickPackShip.ScanExtension
{
    [PXOverride]
    public virtual ScanMode<PickPackShip> DecorateScanMode(
        ScanMode<PickPackShip> original,
        Func<ScanMode<PickPackShip>, ScanMode<PickPackShip>> base_DecorateScanMode)
    {
        return base_DecorateScanMode(original);
    }
}
```

6. Search for the mode, and patch its `CreateTransitions` method via interception functionality.

```
public class LocationGoesLastInPickMode : PickPackShip.ScanExtension
{
    [PXOverride]
    public virtual ScanMode<PickPackShip> DecorateScanMode(
        ScanMode<PickPackShip> original,
        Func<ScanMode<PickPackShip>, ScanMode<PickPackShip>> base_DecorateScanMode)
    {
        var mode = base_DecorateScanMode(original);

        if (mode is PickPackShip.PickMode pickMode)
        {
            pickMode
                // Replaces the previous implementation with a new one.
                .Intercept.CreateTransitions.ByReplace(basis =>
                {
                    // Creates a list of sequential transitions.
                    return basis.StateFlow(flow => flow
                        .From<PickPackShip.PickMode.ShipmentState>()
                        .NextTo<WMSBase.InventoryItemState>()
                        .NextTo<WMSBase.LotSerialState>()
                        .NextTo<WMSBase.ExpireDateState>()
                        .NextTo<WMSBase.LocationState>());
                });
        }

        return mode;
    }
}
```

7. Optional: Extract the exact patching logic to a separate virtual method of the extension so that you can suppress this patch in the future.

```
public class LocationGoesLastInPickMode : PickPackShip.ScanExtension
{
    [PXOverride]
    public virtual ScanMode<PickPackShip> DecorateScanMode(
        ScanMode<PickPackShip> original,
        Func<ScanMode<PickPackShip>, ScanMode<PickPackShip>> base_DecorateScanMode)
```

```
    {
        var mode = base_DecorateScanMode(original);

        if (mode is PickPackShip.PickMode pickMode)
            PatchPickMode(pickMode);

        return mode;
    }

    protected virtual void PatchPickMode(PickPackShip.PickMode pickMode)
    {
        pickMode
            .Intercept.CreateTransitions.ByReplace(basis =>
            {
                return basis.StateFlow(flow => flow
                    .From<PickPackShip.PickMode.ShipmentState>()
                    .NextTo<WMSBase.InventoryItemState>()
                    .NextTo<WMSBase.LotSerialState>()
                    .NextTo<WMSBase.ExpireDateState>()
                    .NextTo<WMSBase.LocationState>());
            });
    }
}
```

8. Fix or suppress the *PX1016* error that is displayed by Acuminator.
9. Build the project, and test the new order of input on the *Pick, Pack, and Ship* (SO302020) form.

Extension of Scan Components: To Add a New Scan Command for an Existing Form

This activity will walk you through the implementation of a new scan command for an existing Acumatica ERP form.

Story

Suppose that you need to add the ability to cancel physical inventory count on the *Scan and Count* (IN305020) form.

Process Overview

You will add an additional scan command component. For details about the implementation of scan commands, see *Barcode Scan Commands: General Information*. You will then override the `DecorateScanMode` method by using the `PXOverride` attribute, and in this method, you will intercept the `CreateCommands` method. You will use the `ByAppend` override strategy, which does not affect the original component.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Enable the following features on the *Enable/Disable Features* (CS100000) form, where they are in the *Inventory and Order Management* group of features:
   - *Multiple Warehouse Locations*

- *Warehouse Management*
- *Fulfillment*
- *Inventory Operations*

Step: Adding a Scan Command

To a scan command, do the following:

1. In the `PhoneRepairShop_Code` Visual Studio project, add a reference to the `PX.BarcodeProcessing.dll` file.
2. Create the `CancelPIAbility` class, and make it `public`.
3. Add the `using` directives shown in the following code to the file.

```
using System;
using PX.Data;
using PX.BarcodeProcessing;
using PX.Common;
using PX.Objects.IN;
using PX.Objects.IN.WMS;
```

4. Make the class an extension of the `PX.Objects.IN.WMS.INScanCount` scan mode class.

```
public class CancelPIAbility : INScanCount.ScanExtension
{
}
```

5. Define a `ScanCommand` component, and implement its members.

```
public class CancelPIAbility : INScanCount.ScanExtension
{
    public sealed class CancelPICommand : INScanCount.ScanCommand
    {
        // The code to be scanned to execute the command
        public override string Code => "CANCEL*PI";
        // The name of the PXAction instance that is created for the command
        public override string ButtonName => "ScanCancelPI";
        // The display name of the button for the PXAction instance that is created
 for the command
        public override string DisplayName => Msg.DisplayName;
        // The Boolean value that indicates when the command can be executed
        protected override bool IsEnabled => Basis.DocumentIsEditable;

        // The logic that is executed for the command
        protected override bool Process()
        {
            var countReview = PXGraph.CreateInstance<INPIReview>();
            countReview.PIHeader.Current =
 countReview.PIHeader.Search<INPIHeader.pIID>(Basis.Document.PIID);
            countReview.cancelPI.Press();

            // Clear the screen
            Basis.CurrentMode.Reset(fullReset: true);
            // Inform the user
            Basis.ReportInfo(Msg.Success);

            return true;
```

```
        }

        [PXLocalizable]
        public abstract class Msg
        {
            public const string DisplayName = "Cancel Count";
            public const string Success = "Physical inventory count has been
 canceled.";
        }
    }
}
```

6. In the `CancelPIAbility` class, override the `DecorateScanMode` method.

```
public class CancelPIAbility : INScanCount.ScanExtension
{
    [PXOverride]
    public virtual ScanMode<INScanCount> DecorateScanMode(
        ScanMode<INScanCount> original,
        Func<ScanMode<INScanCount>, ScanMode<INScanCount>> base_DecorateScanMode)
    {
        return base_DecorateScanMode(original);
    }

    ...
}
```

7. Search for the mode you are looking for, and patch its `CreateCommands` method via the interception functionality.

```
public class CancelPIAbility : INScanCount.ScanExtension
{
    [PXOverride]
    public virtual ScanMode<INScanCount> DecorateScanMode(
        ScanMode<INScanCount> original,
        Func<ScanMode<INScanCount>, ScanMode<INScanCount>> base_DecorateScanMode)
    {
        var mode = base_DecorateScanMode(original);

        if (mode is INScanCount.CountMode countMode)
        {
            countMode
                .Intercept.CreateCommands.ByAppend(() =>
                {
                    // Though you have only one new command to append, you must wrap
 it in an array.
                    return new[] { new CancelPICommand() };
            });
        }

        return mode;
    }

    ...
}
```

8. Optional: Extract the exact patching logic to a separate virtual method of the extension so that you can suppress this patch in the future.

```
public class CancelPIAbility : INScanCount.ScanExtension
{
    [PXOverride]
    public virtual ScanMode<INScanCount> DecorateScanMode(
        ScanMode<INScanCount> original,
        Func<ScanMode<INScanCount>, ScanMode<INScanCount>> base_DecorateScanMode)
    {
        var mode = base_DecorateScanMode(original);
        if (mode is INScanCount.CountMode countMode)
            PatchCountMode(countMode);
        return mode;
    }

    protected virtual void PatchCountMode(INScanCount.CountMode countMode)
    {
        countMode
            .Intercept.CreateCommands.ByAppend(() =>
            {
                return new[] { new CancelPICommand() };
            });
    }

    ...
}
```

9. Fix or suppress the *PX1016* error that is displayed by Acuminator.
10. Build the project, and test the new command on the *Scan and Count* (IN305020) form.

Extension of Scan Components: To Implement Additional Entity Validation

This activity will walk you through the implementation of additional entity validation.

Story

Suppose that you need to forbid the receipt of items whose country of origin is Antarctica on the *Receive and Put Away* (PO302020) form.

Process Overview

You will override the `DecorateScanState` method by using the `PXOverride` attribute, and in this method, you will intercept the `Validation` method. You need to add a validation method after all the already existing ones. Therefore, you will intercept the `Validation` method by using the `ByAppend` strategy.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.

2. Enable the following features on the *Enable/Disable Features* (CS100000) form, where they are in the *Inventory and Order Management* group of features:
   - *Multiple Warehouse Locations*
   - *Warehouse Management*
   - *Receiving*

Step: Implementing Additional Entity Validation

To implement additional entity validation, do the following:

1. In the `PhoneRepairShop_Code` Visual Studio project, add a reference to the `PX.BarcodeProcessing.dll` file.
2. Create the `BoycottAntarcticaItems` class, and make it `public`.
3. Add the `using` directives shown in the following code to the file.

```
using System;
using PX.Data;
using PX.BarcodeProcessing;
using PX.Common;
using PX.Objects.PO.WMS;
```

4. Make the class an extension of the `PX.Objects.PO.WMS.ReceivePutAway` class.

```
public class BoycottAntarcticaItems : ReceivePutAway.ScanExtension
{
}
```

5. Override the `DecorateScanState` method.

```
public class BoycottAntarcticaItems : ReceivePutAway.ScanExtension
{
    [PXOverride]
    public virtual ScanState<ReceivePutAway> DecorateScanState(
        ScanState<ReceivePutAway> original,
        Func<ScanState<ReceivePutAway>, ScanState<ReceivePutAway>>
 base_DecorateScanState)
    {
        return base_DecorateScanState(original);
    }
}
```

6. Patch the logic of the component by using the interceptor functionality.

```
public class BoycottAntarcticaItems : ReceivePutAway.ScanExtension
{
    [PXOverride]
    public virtual ScanState<ReceivePutAway> DecorateScanState(
        ScanState<ReceivePutAway> original,
        Func<ScanState<ReceivePutAway>,
            ScanState<ReceivePutAway>> base_DecorateScanState)
    {
        var state = base_DecorateScanState(original);

        if (state is ReceivePutAway.InventoryItemState itemState &&
            itemState.ModeCode == ReceivePutAway.ReceiveMode.Value)
        {
```

```
            itemState
                .Intercept.Validate.ByAppend((basis, item) =>
                {
                    (var xref, var inventory) = item;
                if (inventory.CountryOfOrigin == "AQ")
                        return Validation.Fail(Msg.CannotReceiveItem,
                            inventory.InventoryCD);

                    return Validation.Ok;
                });
        }

        return state;
    }

    [PXLocalizable]
    public abstract class Msg
    {
        public const string CannotReceiveItem =
            "The {0} item cannot be received.";
    }
}
```

7. Optional: Extract the exact patching logic to a separate virtual method of the extension so that you can suppress this patch in the future.

```
public class BoycottAntarcticaItems : ReceivePutAway.ScanExtension
{
    [PXOverride]
    public virtual ScanState<ReceivePutAway> DecorateScanState(
        ScanState<ReceivePutAway> original,
        Func<ScanState<ReceivePutAway>,
            ScanState<ReceivePutAway>> base_DecorateScanState)
    {
        var state = base_DecorateScanState(original);

        if (state is ReceivePutAway.InventoryItemState itemState
            && itemState.ModeCode == ReceivePutAway.ReceiveMode.Value)
            PatchInventoryItemStateInReceiveMode(itemState);

        return state;
    }

    protected virtual void PatchInventoryItemStateInReceiveMode(
        ReceivePutAway.InventoryItemState itemState)
    {
        itemState
            .Intercept.Validate.ByAppend((basis, item) =>
            {
                (var xref, var inventory) = item;
                if (inventory.CountryOfOrigin == "AQ")
                    return Validation.Fail(Msg.CannotReceiveItem,
                        inventory.InventoryCD);

                return Validation.Ok;
            });
    }
```

```
    [PXLocalizable]
    public abstract class Msg
    {
        public const string CannotReceiveItem =
            "The {0} item cannot be received.";
    }
}
```

8. Fix or suppress the *PX1016* error that is displayed by Acuminator.
9. Build the project, and test the custom behavior on the *Receive and Put Away* (PO302020) form.

Extension of Scan Components: To Add an Alternate Way to Search for Entities

This activity will walk you through the implementation of an alternate way to search for entities.

Story

Suppose that on the *Receive and Put Away* (PO302020) form, you need to be able to search purchase receipts by their date because you have only one unreleased purchase receipt per day.

Process Overview

You will override the `DecorateScanState` method by using the `PXOverride` attribute, and in this method, you will intercept the `HandleAbsence` method by using the `ByAppend` strategy. You will not override the `GetByBarcode` method, but instead will override the `HandleAbsence` method because it contains the logic of entity searching if the entity has not been found by the `GetByBarcode` method.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Enable the following features on the *Enable/Disable Features* (CS100000) form, where they are in the *Inventory and Order Management* group of features:
   - *Multiple Warehouse Locations*
   - *Warehouse Management*
   - *Receiving*

Step: Adding an Alternate Way to Search for Entities

To add another way to search for entities, do the following:

1. In the `PhoneRepairShop_Code` Visual Studio project, add a reference to the `PX.BarcodeProcessing.dll` file.
2. Create the `SearchReceiptsByDate` class, and make it `public`.
3. Add the `using` directives shown in the following code to the file.

```
using System;
using PX.Data;
```

```
using PX.BarcodeProcessing;
using PX.Objects.PO.WMS;
using PX.Objects.PO;
using PX.Data.BQL.Fluent;
using PX.Objects.AP;
using PX.Data.BQL;
```

4. Make the class an extension of the `PX.Objects.PO.WMS.ReceivePutAway` class.

```
public class SearchReceiptsByDate : ReceivePutAway.ScanExtension
{
}
```

5. Override the `DecorateScanState` method.

```
public class SearchReceiptsByDate : ReceivePutAway.ScanExtension
{
    [PXOverride]
    public virtual ScanState<ReceivePutAway> DecorateScanState(
        ScanState<ReceivePutAway> original,
        Func<ScanState<ReceivePutAway>,
            ScanState<ReceivePutAway>> base_DecorateScanState)
    {
        return base_DecorateScanState(original);
    }
}
```

6. Search for the receipt input state of the receive mode.

```
public class SearchReceiptsByDate : ReceivePutAway.ScanExtension
{
    [PXOverride]
    public virtual ScanState<ReceivePutAway> DecorateScanState(
        ScanState<ReceivePutAway> original,
        Func<ScanState<ReceivePutAway>,
            ScanState<ReceivePutAway>> base_DecorateScanState)
    {
        var state = base_DecorateScanState(original);

        if (state is ReceivePutAway.ReceiveMode.ReceiptState receiptState)
        {
        }

        return state;
    }
}
```

7. Patch the logic of the component by using the interceptor functionality.

```
public class SearchReceiptsByDate : ReceivePutAway.ScanExtension
{
    [PXOverride]
    public virtual ScanState<ReceivePutAway> DecoracteScanState(
        ScanState<ReceivePutAway> original,
        Func<ScanState<ReceivePutAway>,
            ScanState<ReceivePutAway>> base_DecoracteScanState)
    {
        var state = base_DecoracteScanState(original);
```

```
        if (state is ReceivePutAway.ReceiveMode.ReceiptState receiptState)
        {
            receiptState
                .Intercept.HandleAbsence.ByAppend((basis, barcode) =>
                {
                    if (DateTime.TryParse(barcode.Trim(), out var date))
                    {
                        POReceipt receiptByDate =
                            SelectFrom<POReceipt>.
                            LeftJoin<Vendor>.On<POReceipt.vendorID.
                                IsEqual<Vendor.bAccountID>>.SingleTableOnly.
                            Where<
                                // The date parameter
                                POReceipt.receiptDate.IsEqual<@P.AsDateTime>.
                                // Other conditions are for narrowing the scope
                                And<POReceipt.released.IsEqual<False>>.
                                And<POReceipt.hold.IsEqual<False>>.

 And<POReceipt.receiptType.IsEqual<POReceiptType.poreceipt>>.
                                And<
                                    Vendor.bAccountID.IsNull.
                                    Or<Match<Vendor,
 AccessInfo.userName.FromCurrent>>>>.
                            View.ReadOnly.Select(basis, date);

                    if (receiptByDate != null)
                            return AbsenceHandling.ReplaceWith(receiptByDate);
                    }

                    return AbsenceHandling.Skipped;
            });
        }

        return state;
    }
}
```

8. Optional: Extract the exact patching logic to a separate virtual method of the extension for being able to suppress this patch in the future.

```
public class SearchReceiptsByDate : ReceivePutAway.ScanExtension
{
    [PXOverride]
    public virtual ScanState<ReceivePutAway> DecorateScanState(
        ScanState<ReceivePutAway> original,
        Func<ScanState<ReceivePutAway>,
            ScanState<ReceivePutAway>> base_DecorateScanState)
    {
        var state = base_DecorateScanState(original);

        if (state is ReceivePutAway.ReceiveMode.ReceiptState receiptState)
            PatchReceiptStateInReceiveMode(receiptState);

        return state;
    }
```

```
    protected virtual void PatchReceiptStateInReceiveMode(
        ReceivePutAway.ReceiveMode.ReceiptState receiptState)
    {
        receiptState
            .Intercept.HandleAbsence.ByAppend((basis, barcode) =>
            {
                if (DateTime.TryParse(barcode.Trim(), out var date))
                {
                    POReceipt receiptByDate =
                        SelectFrom<POReceipt>.
                        LeftJoin<Vendor>.On<POReceipt.vendorID.
                            IsEqual<Vendor.bAccountID>>.SingleTableOnly.
                        Where<
                            POReceipt.receiptDate.IsEqual<@P.AsDateTime>.
                            And<POReceipt.receiptType.
                                IsEqual<POReceiptType.poreceipt>>.
                            And<
                                Vendor.bAccountID.IsNull.
                                Or<Match<Vendor, AccessInfo.userName.FromCurrent>>>>.
                        View.ReadOnly.Select(basis, date);

                    if (receiptByDate != null)
                        return AbsenceHandling.ReplaceWith(receiptByDate);
                }

                return AbsenceHandling.Skipped;
        });
    }
}
```

9. Fix or suppress the *PX1016* error that is displayed by Acuminator.
10. Build the project, and test the custom behavior on the *Receive and Put Away* (PO302020) form.

Customizing a Barcode-Driven Form

In this chapter, you will learn how to customize a barcode-driven form.

Customization of a Barcode-Driven Form: General Information

The barcode-driven engine uses a component-based system to make its customization easier.

Learning Objectives

In this chapter, you will learn how to customize a barcode-driven form.

Applicable Scenarios

You customize a barcode-driven form when you need to modify the behavior of this form.

Customization Approach

When you customize a barcode-driven form, you do the following:

1. Find the barcode scan class you want to customize, and create an extension for it by using the `ScanExtension` component of the chosen class. For details about `ScanExtension`, see *Extension of Scan Components: Scan Extensions*.
2. Decide which part of the logic of the form you want to change. Depending on the part you need to change, use the recommendations in the following topics:
   - If you need to customize the logic of the barcode scan class, see *Customization of a Barcode-Driven Form: Barcode Scan Class*
   - If you need to customize a scan component, see *Extension of Scan Components: Customization of Components*

Namespaces

When you are customizing a barcode-driven form, you may need to import the following namespaces.

```
using System;
using System.Linq;
using System.Collections;
using System.Collections.Generic;

using PX.Common;
using PX.Data;
using PX.Data.BQL;
using PX.Data.BQL.Fluent;
using PX.BarcodeProcessing;

using PX.Objects.IN;
using PX.Objects.IN.WMS;
// XX is the name of the module in which the target class is located
using PX.Objects.XX;
using PX.Objects.XX.WMS;
```

Generic Extensions

If you want to create a generic extension for a barcode-driven form, you use the following template. For details on generic extensions, see *Generic Graph Extensions: General Information*.

```
public abstract class SomeGenericWMSExtension<TScanBasis, TScanGraph> :
    PXGraphExtension<TScanBasis, TScanGraph>
    where TScanBasis : WarehouseManagementSystem<TScanBasis, TScanGraph>
    where TScanGraph : PXGraph, new()
{
    // Is necessary for consistency
    protected TScanBasis Basis => Base1;

    // Gets an instance of this extension within its components
    public static SomeGenericWMSExtension<TScanBasis, TScanGraph>
        GetSelf(TScanBasis basis) =>
            basis.Get<SomeGenericWMSExtension<TScanBasis, TScanGraph>>();

    // Adds customization logic
    [PXOverride]
    public virtual ScanState<TScanBasis> DecorateScanState(
        ScanState<TScanBasis> original,
        Func<ScanState<TScanBasis>, ScanState<TScanBasis>> base_DecorateScanState)
    {
        ...
    }
}
```

Customization of a Barcode-Driven Form: Barcode Scan Class

If you need to override the logic of the barcode scan class, you override the member of the class by using the `PXOverride` attribute, as shown in the following code.

```
public class MyScanExtension : SomeWMS.ScanExtension
{
    [PXOverride]
    public virtual int Foo(string input, Func<string, int> base_Foo)
    {
        // Your logic that overrides the SomeWMS.Foo(string) method
    }
}
```

Customization of a Barcode-Driven Form: To Handle Input Without Component Usage

This activity will walk you through the implementation of input handling without component usage.

Story

Suppose that you want to add the ability to append text to the note of the document that the user is currently working with on a barcode-driven form. Also, you want to make this ability generic for any barcode-driven form.

Process Overview

You will create a generic extension for the barcode-driven engine and implement custom logic in it.

System Preparation

Before you begin performing the step of this activity, do the following:

1. Prepare an Acumatica ERP instance by performing the *Test Instance: To Deploy an Instance* prerequisite activity.
2. Enable the following features on the *Enable/Disable Features* (CS100000) form, where they are in the *Inventory and Order Management* group of features:
   - *Multiple Warehouse Locations*
   - *Warehouse Management*
   - *Fulfillment*

Step: Handling Input Without Component Usage

To modify the barcode-driven engine to be able to append a text to a note, do the following:

1. In the `PhoneRepairShop_Code` Visual Studio project, add a reference to the `PX.BarcodeProcessing.dll` file.
2. Create the `WMSNoteAppender` class.
3. Add the `using` directives shown in the following code to the file of the class.

```
using System;
using PX.Data;
using PX.BarcodeProcessing;
using PX.Common;
using PX.Objects.IN;
using PX.Objects.IN.WMS;
```

4. Create a generic extension for the barcode-driven engine.

```
public abstract class WMSNoteAppender<TScanBasis, TScanGraph> :
        PXGraphExtension<TScanBasis, TScanGraph>
    where TScanBasis : BarcodeDrivenStateMachine<TScanBasis, TScanGraph>
    where TScanGraph : PXGraph, new()
    {
        protected TScanBasis Basis => Base1;
    }
```

5. Add additional type parameter that will represent the note owner entity.

```
public abstract class WMSNoteAppender<TDocument, TScanBasis, TScanGraph> :
    PXGraphExtension<TScanBasis, TScanGraph>
where TDocument : PXBqlTable, IBqlTable, new()
where TScanBasis : BarcodeDrivenStateMachine<TScanBasis, TScanGraph>
where TScanGraph : PXGraph, new()
```

```
{
    protected TScanBasis Basis => Base1;
}
```

6. Override the `ProcessCustomScan` method by using the `PXOverride` attribute.

```
public abstract class WMSNoteAppender<TDocument, TScanBasis, TScanGraph> :
    PXGraphExtension<TScanBasis, TScanGraph>
where TDocument : class, IBqlTable, new()
where TScanBasis : BarcodeDrivenStateMachine<TScanBasis, TScanGraph>
where TScanGraph : PXGraph, new()
{
    protected TScanBasis Basis => Base1;

    [PXOverride]
    public virtual bool? ProcessCustomScan(
        string barcode,
        Func<string, bool?> base_ProcessCustomScan)
    {
        return base_ProcessCustomScan(barcode);
    }
}
```

7. Add the logic of the handler.

```
public abstract class WMSNoteAppender<TDocument, TScanBasis, TScanGraph> :
    PXGraphExtension<TScanBasis, TScanGraph>
where TDocument : class, IBqlTable, new()
where TScanBasis : BarcodeDrivenStateMachine<TScanBasis, TScanGraph>
where TScanGraph : PXGraph, new()
{
    protected TScanBasis Basis => Base1;

    public const string NotePrefix = "note:";

    [PXOverride]
    public virtual bool? ProcessCustomScan(string barcode,
        Func<string, bool?> base_ProcessCustomScan)
    {
        if (barcode.StartsWith(NotePrefix))
        {
            string value = barcode.Substring(NotePrefix.Length).Trim();

            if (!string.IsNullOrEmpty(value))
            {
                PXCache<TDocument> docCache = Basis.Graph.Caches<TDocument>();
                // Use an abstract function to get the entity.
                var document = GetNoteOwnerEntity();

                if (document != null && docCache.Fields.Contains("NoteID"))
                {
                    string existingValue = PXNoteAttribute.GetNote(docCache,
 document);
                    string newLine =

 $"[{Basis.Graph.Accessinfo.UserName}@{DateTime.Now.ToShortDateString()}]: {value}";

                    PXNoteAttribute.SetNote(docCache, document,
```

```
                        string.IsNullOrEmpty(existingValue)
                            ? newLine
                            : existingValue + Environment.NewLine + newLine);

                    Basis.SaveChanges();
                    Basis.ReportInfo(Msg.Success);

                    return true;
                }
                else
                {
                    Basis.ReportError(Msg.Fail);

                    return false;
                }
            }
        }

        return base_ProcessCustomScan(barcode);
    }

    protected abstract TDocument GetNoteOwnerEntity();

    [PXLocalizable]
    public abstract class Msg
    {
        public const string Success = "Your note was successfully added.";
        public const string Fail = "The system was not able to add your note.";
    }
}
```

The `ProcessCustomScan` method will be called before the current `ScanState` processor attempts to process a barcode only if none of `ScanCommand` processors and none of `ScanRedirect` processors are able to handle the scanned barcode.

The return value of the `ProcessCustomScan` method has the `bool?` type. If the method returns `null`, the scanned barcode will be passed to the current `ScanState` processor. Otherwise, the returned value of the `ProcessCustomScan` method would show whether the custom scan handler succeeded or failed.

You use the *note:text to append* pattern to recognize the wish of a user to append text to the note of the current entity.

8. Add the end extension for the specific barcode scan class.

```
public class ItemNoteAttacher : WMSNoteAppender<InventoryItem,
    InventoryItemLookup, InventoryItemLookup.Host>
{
    protected override InventoryItem GetNoteOwnerEntity() =>
        Basis.InventoryItem.Select(); // decide how to get the underlying entity
}
```

9. Fix or suppress the *PX1016* errors that are displayed by Acuminator.
10. Build the project and test the custom functionality.